

**TEMPLATE
PROJECT WORK**

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (<i>pari a circa Minimo 12 – Massimo 20 pagine</i>)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Giuseppe Carbone
Numero di matricola	0312301748
Tema n. (Indicare il numero del tema scelto):	1
Titolo del tema (Indicare il titolo del tema scelto):	La digitalizzazione dell'impresa
Traccia del PW n. (Indicare il numero della traccia scelta):	PW16
Titolo della traccia (Indicare il titolo della traccia scelta):	Sviluppo di una applicazione full-stack API-based per un'organizzazione del settore sanitario
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Vita Serena: Piattaforma digitale per la gestione delle prenotazioni di un poliambulatorio

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

La realizzazione dell'elaborato "Vita Serena" ha richiesto l'integrazione sistematica di competenze acquisite lungo l'intero percorso di studio in Informatica per le Aziende Digitali (L-31).

Di seguito espongo, per ciascun insegnamento, non soltanto l'ambito disciplinare di provenienza, ma il collegamento puntuale con le scelte implementative operate nel progetto.

Dagli insegnamenti di **Programmazione 1** e **Programmazione 2** ho acquisito i fondamenti della programmazione strutturata e, progressivamente, del paradigma orientato agli oggetti in Python.

Queste competenze sono state determinanti nella scrittura del backend con FastAPI: ogni entità del dominio applicativo, dalle credenziali di registrazione ai dati di una prenotazione, è stata modellata attraverso classi che estendono `pydantic.BaseModel`, sfruttando meccanismi propri della programmazione a oggetti quali l'ereditarietà, l'incapsulamento degli attributi tipizzati e l'uso di metodi di classe per la validazione personalizzata tramite il decoratore `@field_validator`.

La comprensione dei principi di astrazione e di riuso del codice, maturata in questi due corsi, mi ha inoltre guidato nella progettazione di funzioni helper condivise tra più endpoint, come `get_paziente_id()` e `get_medico_id()`, che centralizzano la logica di risoluzione dell'identità applicativa a partire dall'identificativo dell'utente autenticato, evitando duplicazioni di codice e riducendo la superficie di errore.

Dall'insegnamento di **Basi di Dati** ho applicato in modo diretto i concetti di progettazione relazionale, normalizzazione e integrità referenziale.

Il modello logico del database è stato costruito seguendo un processo di normalizzazione fino alla terza forma normale, evitando ridondanze informative e garantendo che ogni attributo non chiave dipendesse funzionalmente dall'intera chiave primaria della propria relazione.

La scelta di adottare un pattern di ereditarietà per tabelle separate tra `utenti` e le sue specializzazioni (`pazienti`, `medici`) discende proprio dai principi di normalizzazione studiati in questo corso, così come la definizione di vincoli di integrità referenziale tramite chiavi esterne con clausole `ON DELETE` esplicite, utilizzate ad esempio nella relazione tra `referti` e `medici` per gestire il caso in cui un medico venga rimosso dal sistema pur mantenendo lo storico clinico dei pazienti.

Ho inoltre applicato competenze di scrittura di query SQL complesse, necessarie ad esempio per ricostruire, in un'unica interrogazione, l'elenco degli appuntamenti con i dati anagrafici di paziente e medico, la specialità di riferimento e gli orari dello slot prenotato.

Dall'insegnamento di **Ingegneria del Software** ho applicato i principi di modularità, separazione delle responsabilità e documentazione tecnica del codice.

Il progetto è organizzato secondo una chiara separazione tra livello di presentazione, livello applicativo e livello dati, e ogni file JavaScript del frontend è dedicato a una singola dashboard, riflettendo il principio di singola responsabilità (Single Responsibility Principle).

L'approccio incrementale adottato nello sviluppo, articolato in otto fasi distinte con obiettivi progressivamente più complessi, riflette inoltre le metodologie agili di sviluppo software studiate in questo corso, in particolare la pratica di consegnare a ogni iterazione un incremento funzionante e verificabile del sistema, anziché tentare una progettazione monolitica e sequenziale dell'intera applicazione.

Dall'insegnamento di **Reti di Calcolatori e Cybersecurity** ho tratto la comprensione approfondita del protocollo HTTP, della semantica dei metodi GET, POST, PATCH e DELETE, e dei principi fondamentali di sicurezza applicativa.

Questa conoscenza mi è stata molto utile nella fase di audit di sicurezza del progetto, durante la quale ho identificato e corretto diverse vulnerabilità di classe IDOR, implementando controlli di autorizzazione a livello di singolo endpoint, introducendo un middleware per l'iniezione di header di sicurezza HTTP e applicando tecniche di sanitizzazione dell'output per prevenire attacchi di tipo Cross-Site Scripting.

L'hashing delle password tramite la libreria bcrypt, con l'utilizzo di un salt casuale generato automaticamente a ogni chiamata, rappresenta un'ulteriore applicazione diretta dei principi di sicurezza crittografica trattati in questo insegnamento.

Dall'insegnamento di **Architettura dei Calcolatori** ho consolidato la comprensione del modello client-server e della comunicazione tra processi distinti, fondamentale per interpretare correttamente la separazione tra il processo del server FastAPI, in ascolto sulla porta 8000, e il processo del browser che esegue il codice JavaScript lato client, comunicando esclusivamente attraverso lo scambio di messaggi HTTP serializzati in formato JSON.

Dall'insegnamento di **Strategia, Organizzazione e Marketing** ho tratto la capacità di analizzare i processi organizzativi di un'impresa del settore sanitario, identificando le inefficienze tipiche della gestione manuale delle prenotazioni, quali la doppia prenotazione dello stesso slot, la difficoltà di coordinamento tra le agende di più medici e l'assenza di un sistema di conferma automatica della presenza del paziente. Questa analisi mi ha permesso di tradurre esigenze organizzative concrete in requisiti funzionali precisi, quali il sistema di check-in in-app e la gestione centralizzata delle disponibilità mediche.

Dall'insegnamento di **Diritto per le Aziende Digitali** ho acquisito la consapevolezza giuridica necessaria per affrontare correttamente il trattamento di dati personali e sanitari all'interno dell'applicazione. Le nozioni relative al Regolamento (UE) 2016/679, e in particolare agli articoli 5, 9 e 25, hanno guidato direttamente la progettazione della funzionalità di Dossier Sanitario Condiviso, basata sul principio di minimizzazione dei dati e sul consenso esplicito e revocabile dell'interessato, così come la redazione dell'informativa privacy pubblicata nella pagina dedicata dell'applicazione.

Dall'insegnamento di **Calcolo delle Probabilità e Statistica** ho tratto le basi metodologiche per l'interpretazione e la presentazione delle statistiche aggregate mostrate nella dashboard amministrativa, quali il numero totale di appuntamenti, gli appuntamenti del giorno corrente e il numero di prenotazioni in attesa di conferma, dati che, seppur descrittivi nella loro forma attuale, costituiscono la base concettuale per eventuali sviluppi futuri orientati all'analisi predittiva dei flussi di prenotazione.

Dall'insegnamento di **Tecnologie Web** ho acquisito le competenze fondamentali sui protocolli e sugli standard che regolano il funzionamento del World Wide Web, in particolare l'architettura del protocollo HTTP, la struttura semantica dell'HTML5, i fogli di

stile a cascata (CSS3) e le tecniche di manipolazione dinamica del DOM tramite JavaScript.

Queste conoscenze sono state applicate direttamente nello sviluppo dell'intero frontend dell'applicazione, realizzato in HTML, CSS e JavaScript Vanilla senza l'ausilio di framework esterni, come richiesto dalla traccia del Project Work.

In particolare, ho applicato i principi di comunicazione asincrona client-server studiati in questo corso attraverso l'uso sistematico dell'API Fetch, che consente al browser di effettuare richieste HTTP verso il backend senza necessità di ricaricare la pagina, garantendo un'esperienza utente fluida e reattiva su tutte le quattro dashboard del sistema.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato

(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Lo sviluppo dell'elaborato ha seguito un approccio incrementale, aggiungendo funzionalità in modo progressivo e verificandone il funzionamento a ogni passo.

Fase 1:

Analisi dei requisiti e progettazione (circa 3 giorni)

È stata analizzata la traccia e identificato lo scenario d'uso: un poliambulatorio privato con quattro tipologie di utenti.

Sono stati definiti i casi d'uso per ogni ruolo e progettato lo schema ER del database.

La difficoltà principale è stata bilanciare semplicità e completezza del modello dati, in particolare per la gestione delle disponibilità orarie dei medici.

Fase 2:

Setup dell'ambiente e autenticazione (1 giorno)

Configurazione dell'ambiente di sviluppo con Python, FastAPI, MySQL tramite XAMPP e struttura delle cartelle del progetto.

Implementazione del sistema di login con redirect per ruolo, registrazione pazienti e sistema di primo accesso con password monouso per medici e segreterie.

La difficoltà incontrata riguardava l'hashing delle password con bcrypt e la gestione del flusso di primo accesso.

Fase 3:

Dashboard Admin (2 giorni)

Sviluppo del pannello amministratore con statistiche generali, gestione CRUD di medici, segreterie e specialità, calendario disponibilità e visualizzazione appuntamenti.

È stata qui introdotta la funzione helper `serializza_righe()` per risolvere il problema della serializzazione JSON degli oggetti data/timedelta restituiti da MySQL.

Fase 4:

Dashboard Medico (2 giorni)

Implementazione del calendario visuale mensile con indicatori per giorno, pannello laterale con bottoni cliccabili in tre stati (vuoto, libero, prenotato) e gestione degli appuntamenti.

La sfida principale è stata il calcolo dinamico dell'ora di fine slot in base alla durata configurata per ciascun medico.

Fase 5:

Dashboard Paziente (2 giorni)

Sviluppo del wizard di prenotazione in 4 step (specialità → medico → slot → conferma), gestione appuntamenti, storico visite e profilo personale.

È stata curata l'esperienza utente con animazioni CSS e feedback visivi chiari.

Fase 6:

Dashboard Segreteria (3 giorni)

Implementazione delle sei sezioni della segreteria: riepilogo giornaliero, lista appuntamenti con filtri multipli, wizard di prenotazione manuale, gestione slot medici con bottoni ora, ricerca pazienti.

Aggiunta del sistema di check-in in-app come alternativa alle notifiche email.

Fase 7:

Funzionalità avanzate e rifinitura (2 giorni)

Aggiunta dello stato "completato" per gli appuntamenti, note visita del medico, durata slot configurabile per medico sia dall'admin che dal medico stesso, pagina 404 personalizzata, validazione date passate e gestione del link "Password dimenticata".

Fase 8:

Gestione referti, compliance GDPR e hardening di sicurezza (2 giorni)

Implementazione del modulo referti: il medico redige il referto per le visite completate e il paziente lo consulta nella propria area riservata.

Aggiunta della compliance GDPR (informativa privacy per dati sanitari ex art. 9, cookie banner, footer con dati societari) e audit di sicurezza sistematico: controlli anti-IDOR su tutti gli endpoint sensibili, sanitizzazione XSS, security headers HTTP (HSTS, CSP, X-Frame-Options), validazione server-side di date e slot duplicati.

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Per la realizzazione dell'elaborato ho utilizzato strumenti software interamente gratuiti e open source: Python 3 con il framework FastAPI e la libreria Pydantic per il backend, PyMySQL per l'accesso ai dati, MySQL amministrato tramite XAMPP con il client grafico Beekeeper Studio, impiegato per la progettazione dello schema, l'esecuzione delle query di verifica e l'ispezione diretta dei dati durante lo sviluppo, la libreria bcrypt per l'hashing delle credenziali, e gli strumenti di sviluppo integrati nel browser (DevTools) per il debug del frontend e l'ispezione delle chiamate HTTP.

Quanto alle risorse documentali, ho fatto riferimento in via primaria alla documentazione ufficiale di FastAPI e Pydantic per la progettazione degli endpoint e dei modelli di validazione, alla Mozilla Developer Network (MDN) per HTML5, CSS3, JavaScript e API Fetch, alla dissertazione dottorale di Roy Fielding (2000) per i vincoli architetturali REST, al testo del Regolamento (UE) 2016/679 consultato su EUR-Lex per gli aspetti di

protezione dei dati, e alle linee guida OWASP per la configurazione degli header di sicurezza e la prevenzione di XSS e IDOR.

La principale difficoltà nel reperimento delle risorse ha riguardato la serializzazione JSON dei tipi data/timedelta restituiti da MySQL, non trattata organicamente nella documentazione: è stata superata incrociando la documentazione di PyMySQL con discussioni tecniche della community, giungendo alla funzione helper serializza_righe() descritta nelle fasi di lavoro. Il codice sorgente completo del progetto, frontend e backend, è disponibile al repository pubblico:

<https://github.com/zeniith14/Project-Work/>.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

L'elaborato risponde in maniera puntuale e circostanziata a quanto richiesto dalla Traccia PW 16, che prevede lo sviluppo di un'applicazione web full-stack basata su architettura API REST per un'organizzazione del settore sanitario.

Di seguito espongo in dettaglio ciascun obiettivo raggiunto, motivando le scelte progettuali sottostanti.

Obiettivo 1: Implementazione del frontend con HTML, CSS e JavaScript La traccia richiede espressamente la realizzazione della parte frontend utilizzando HTML, CSS e JavaScript, lasciando libertà sull'eventuale adozione di un framework.

Ho scelto deliberatamente di non utilizzare alcun framework (React, Vue, Angular o simili), optando per JavaScript Vanilla.

Questa decisione, apparentemente in controtendenza rispetto alle pratiche più diffuse nello sviluppo frontend moderno, è stata motivata da un obiettivo didattico preciso: dimostrare la piena padronanza dei meccanismi nativi del DOM, della gestione degli eventi e della comunicazione asincrona, senza le astrazioni fornite da una libreria terza.

Ho realizzato quattro interfacce utente distinte, una per ciascun ruolo del sistema (amministratore, medico, segreteria, paziente), ciascuna organizzata secondo un pattern di navigazione a sezioni (single page application semplificata), in cui il cambio di vista avviene tramite manipolazione diretta delle classi CSS di visibilità, senza ricaricamento della pagina e senza il supporto di un router lato client.

Obiettivo 2: Sviluppo di un backend API RESTful in un linguaggio object-oriented Il backend, interamente realizzato con FastAPI su Python, espone oltre quaranta endpoint REST organizzati per dominio funzionale: autenticazione, gestione anagrafica, prenotazione visite, gestione delle disponibilità mediche, ciclo di vita degli appuntamenti, sistema di check-in, gestione dei referti e Dossier Sanitario Condiviso.

La traccia richiede l'utilizzo di un linguaggio orientato agli oggetti per la realizzazione del backend: Python soddisfa pienamente questo requisito, e FastAPI ne amplifica il paradigma attraverso l'uso sistematico di classi Pydantic per la definizione dei contratti di richiesta e risposta.

Ogni payload in ingresso viene validato automaticamente contro un modello dichiarativo, secondo un approccio che unisce i principi della programmazione a oggetti (ereditarietà

da BaseModel, incapsulamento degli attributi tipizzati) alla robustezza tipica dei linguaggi fortemente tipizzati, pur mantenendo la flessibilità sintattica propria di Python.

Obiettivo 3: Fornire un servizio significativo per un'organizzazione del settore sanitario. La traccia richiede che l'applicazione fornisca un servizio realmente utile per un'impresa del settore sanitario, citando come esempi la prenotazione di visite e la gestione dei referti.

Ho affrontato entrambi gli esempi proposti, estendendoli con funzionalità che ne aumentano il realismo operativo: il sistema copre l'intero ciclo di vita di un appuntamento, dalla prenotazione online autonoma da parte del paziente, alla conferma o al rifiuto da parte del medico o della segreteria, fino alla conclusione della visita con la redazione di note cliniche e di un referto strutturato.

A questo si aggiunge il sistema di check-in in-app, pensato per ridurre il fenomeno dei mancati presentati (no-show), e il Dossier Sanitario Condiviso, funzionalità che ho introdotto autonomamente per rispondere a un'esigenza reale delle strutture sanitarie multidisciplinari: la necessità di condividere in modo controllato lo storico clinico di un paziente tra più professionisti, nel rispetto della normativa sulla protezione dei dati personali.

Obiettivo 4: Adozione di un'architettura moderna e disaccoppiata. Infine, l'applicazione adotta un'architettura a più livelli, con una netta separazione tra il frontend, responsabile esclusivamente della presentazione e dell'interazione con l'utente, e il backend, responsabile della logica applicativa e dell'accesso ai dati.

Questa separazione, comunicante unicamente attraverso chiamate HTTP asincrone in formato JSON, garantisce non soltanto la conformità al paradigma REST richiesto dalla traccia, ma anche una base architetturale scalabile: la stessa API potrebbe in futuro essere consumata da un client differente, quale un'applicazione mobile nativa, senza alcuna modifica al backend esistente, dimostrando concretamente il valore della separazione delle responsabilità tra i livelli del sistema.

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

Contesto teorico

Le applicazioni web moderne adottano sempre più frequentemente un'architettura a due livelli separati: un frontend che gestisce l'interfaccia utente e un backend che espone i dati tramite API REST.

Questo paradigma, noto come architettura client-server disaccoppiata, permette una maggiore flessibilità, scalabilità e manutenibilità rispetto alle applicazioni monolitiche tradizionali.

Nel settore sanitario, la digitalizzazione dei processi amministrativi come la gestione delle prenotazioni rappresenta un'esigenza sempre più diffusa.

Cliniche, studi medici e poliambulatori si trovano a dover gestire grandi volumi di prenotazioni, garantendo al contempo la privacy dei dati (GDPR) e l'accessibilità del servizio a diverse categorie di utenti.

Contesto applicativo

Il Poliambulatorio Vita Serena è una struttura sanitaria privata multidisciplinare che, prima della digitalizzazione, gestiva le prenotazioni manualmente tramite telefono e agenda cartacea.

Questo sistema comportava inefficienze quali: doppia prenotazione dello stesso slot, difficoltà nel coordinare le disponibilità di più medici, assenza di conferma automatica della presenza del paziente.

L'applicazione sviluppata digitalizza l'intero processo, coinvolgendo quattro categorie di attori:

- **Pazienti:** prenotano autonomamente online, scegliendo specialità, medico e orario
- **Medici** gestiscono la propria agenda settimanale direttamente dalla dashboard
- **Segreteria:** supervisiona, coordina e interviene manualmente quando necessario
- **Amministratore:** monitora le statistiche globali e gestisce l'anagrafica della struttura

Descrizione dei principali aspetti progettuali

(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Di seguito approfondisco ulteriormente alcuni aspetti progettuali dell'applicazione, complementari a quanto già descritto, per fornire una visione più completa delle scelte architetturali operate.

Il pattern architetturale in dettaglio

L'architettura del sistema può essere descritta più precisamente come un'architettura a tre livelli (three-tier architecture), in cui ciascun livello ha una responsabilità circoscritta e comunica esclusivamente con il livello adiacente. Il livello di presentazione, costituito dalle pagine HTML e dagli script JavaScript eseguiti nel browser, non contiene alcuna logica di business: la sua unica responsabilità è raccogliere l'input dell'utente, inviarlo al backend tramite chiamate Fetch asincrone, e renderizzare la risposta ricevuta. Il livello applicativo, implementato in FastAPI, contiene l'intera logica di business del sistema: la validazione dei dati in ingresso, l'applicazione delle regole di autorizzazione, il calcolo degli stati derivati (ad esempio l'orario di fine di uno slot in base alla durata configurata per il medico), e l'orchestrazione delle operazioni sul database. Il livello dati, rappresentato dal database MySQL, si limita alla persistenza strutturata delle informazioni, con l'unico livello di logica delegato al database rappresentato dai vincoli di integrità referenziale (chiavi esterne) e dai vincoli di dominio (colonne di tipo ENUM per gli stati). Questa rigorosa separazione delle responsabilità, oltre a facilitare la manutenibilità del codice, mi ha permesso di testare e correggere ciascun livello in maniera relativamente indipendente durante la fase di sviluppo incrementale.

Gestione dello stato applicativo lato client

A differenza di molte applicazioni web moderne che si affidano a un sistema di sessione lato server basato su cookie firmati o a token JWT con scadenza, ho scelto di implementare un meccanismo di autenticazione più elementare, basato sulla persistenza dei dati dell'utente autenticato nel localStorage del browser dopo una risposta positiva dell'endpoint /api/login. Questa scelta, pur presentando dei limiti in termini di sicurezza che ho discusso e in parte mitigato nella sezione dedicata all'hardening dell'applicazione (introducendo controlli di autorizzazione a livello di singolo endpoint anziché affidarmi a un meccanismo di sessione centralizzato), mi ha consentito di concentrare l'attenzione progettuale sulla comprensione approfondita del ciclo di vita di una richiesta HTTP stateless, requisito concettuale imprescindibile per comprendere appieno il paradigma REST. Ogni pagina protetta verifica all'avvio la presenza e la coerenza del ruolo salvato in localStorage, reindirizzando l'utente alla pagina di login in caso di assenza o incoerenza, un controllo che ho implementato in modo uniforme in ciascuno dei quattro file JavaScript principali del progetto.

Strategia di gestione degli errori e dei codici di stato HTTP

Un aspetto progettuale a cui ho dedicato particolare cura è la coerenza semantica dei codici di stato HTTP restituiti dalle API, elemento spesso trascurato in progetti didattici ma fondamentale per un'interfaccia RESTful ben progettata. Ho adottato la seguente convenzione, applicata sistematicamente a tutti gli endpoint del backend: il codice 200 per le operazioni di lettura e aggiornamento andate a buon fine, il codice 201 concettualmente riservato alle operazioni di creazione (sebbene FastAPI restituisca 200 di default salvo diversa configurazione esplicita), il codice 400 per payload malformati rilevati dalla logica applicativa, il codice 403 per le richieste che violano un controllo di autorizzazione o di titolarità (ownership), il codice 404 per le risorse non trovate, il codice 409 per conflitti di stato (ad esempio il tentativo di prenotare uno slot già occupato), e il codice 422, generato automaticamente da FastAPI, per gli errori di validazione rilevati direttamente dai modelli Pydantic prima ancora che la richiesta raggiunga la funzione dell'endpoint. Questa coerenza ha permesso al frontend di implementare una gestione degli errori uniforme, mostrando all'utente messaggi differenziati a seconda della natura del problema riscontrato, anziché un generico messaggio di errore indistinto.

Considerazioni su prestazioni e scalabilità

Sebbene il progetto sia stato sviluppato e testato in un ambiente locale con un volume di dati contenuto, ho tenuto in considerazione, in fase di progettazione, alcuni aspetti rilevanti per un'eventuale scalabilità futura del sistema. La scelta di PyMySQL con cursori di tipo dizionario, pur comportando un lieve overhead rispetto ai cursori standard, semplifica sensibilmente la manipolazione dei risultati delle query e la loro conversione in strutture JSON serializzabili, riducendo la probabilità di errori di indicizzazione manuale delle colonne. Ho inoltre evitato pattern potenzialmente problematici in termini di prestazioni, quali l'esecuzione di query all'interno di cicli (pattern noto come N+1 query problem), preferendo in ogni caso l'uso di istruzioni SQL con JOIN multiple che recuperano in un'unica interrogazione tutti i dati correlati necessari alla costruzione della risposta, un approccio che riduce il numero di round-trip verso il database e che risulterebbe tanto più vantaggioso quanto maggiore fosse il volume di dati gestito dal sistema in un contesto di produzione reale.

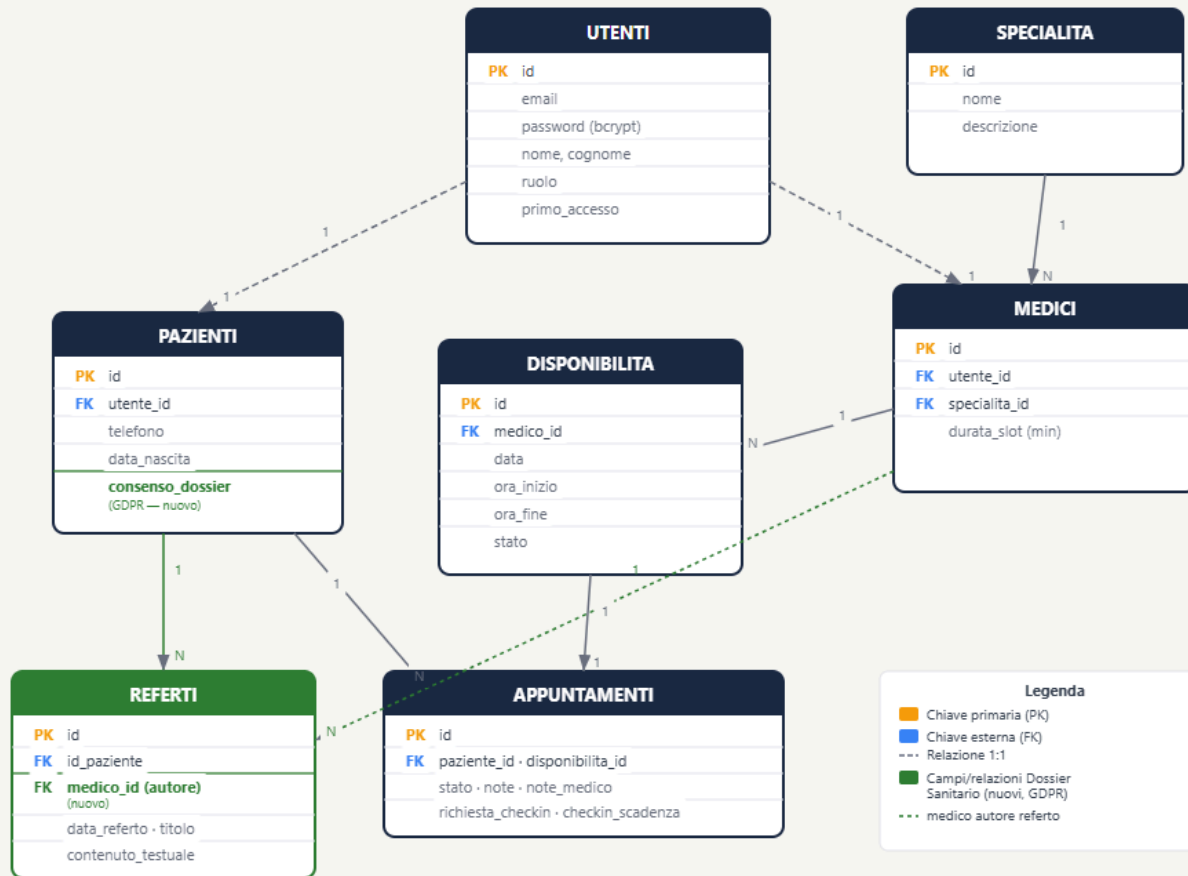
Aspetti di Design (UML ed ER)

Per la progettazione architettuale di "Vita Serena", ho adottato standard di modellazione visuale al fine di mappare con precisione sia la struttura dei dati (Entity-Relationship) sia le interazioni degli attori con il sistema (Use Case Diagram).

Diagramma Entità-Relazione (ER)

Il modello logico del database relazionale è stato progettato per garantire l'integrità dei dati, evitare ridondanze e supportare l'architettura RESTful del backend, ponendo particolare enfasi sulle direttive privacy imposte dal GDPR.

Diagramma ER – Vita Serena



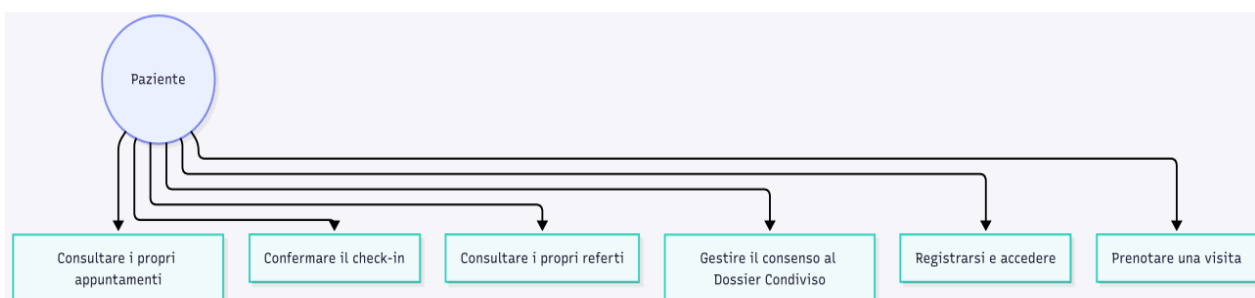
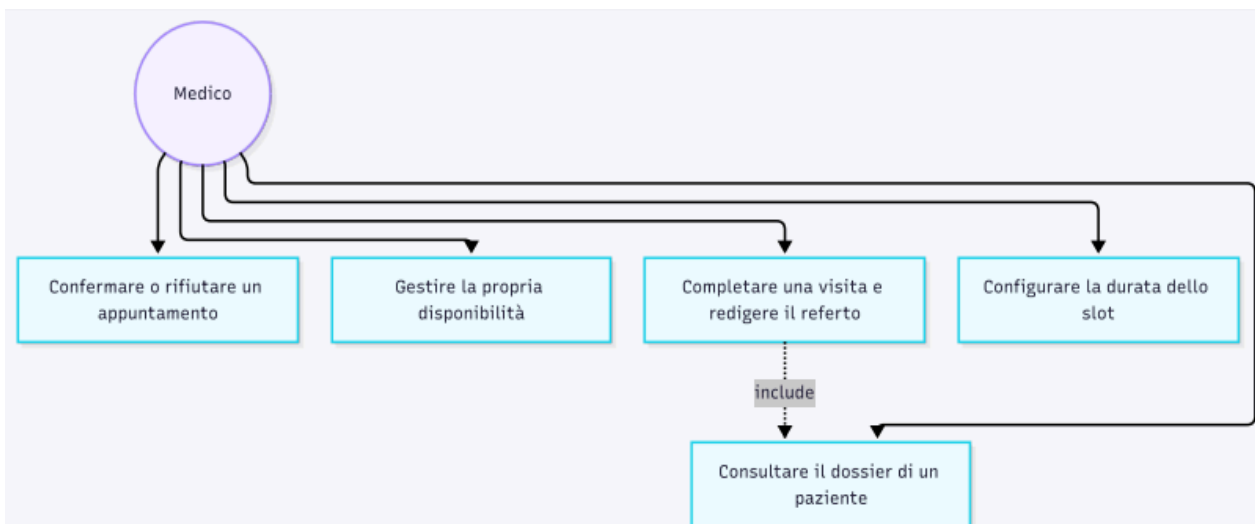
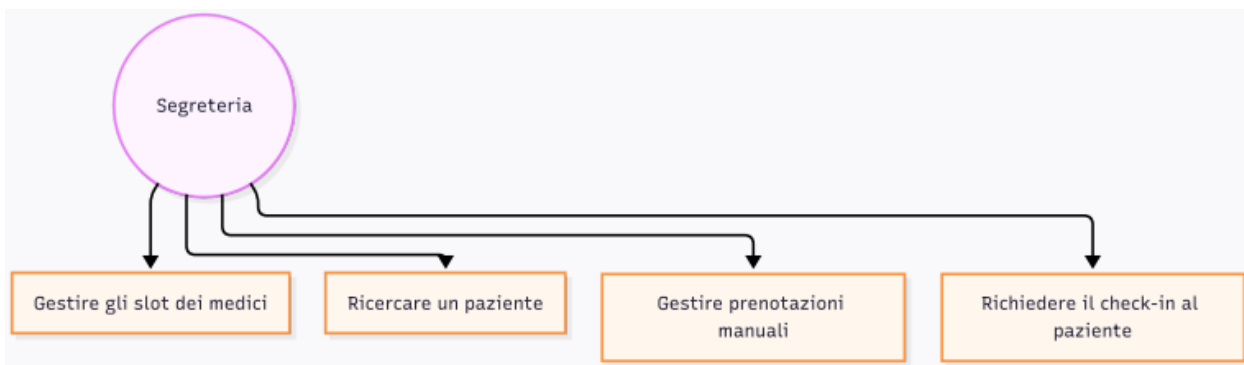
Descrizione del Modello Dati Come si evince dal diagramma, l'architettura del database ruota attorno all'entità centrale **UTENTI**, che centralizza non solo i dati di autenticazione (email, password hashata con bcrypt, ruolo e flag di primo accesso), ma anche l'anagrafica base (nome e cognome). Da questa entità derivano, tramite una relazione 1:1 basata sulla chiave esterna **utente_id**, le tabelle specializzate **PAZIENTI** e **MEDICI**, che contengono le informazioni specifiche del dominio. Questo pattern di ereditarietà garantisce un login centralizzato e una rigorosa normalizzazione.

La gestione delle visite è modellata attraverso la disaccoppiatura tra **DISPONIBILITA** e **APPUNTAMENTI**. Un Medico può definire molteplici slot di disponibilità (relazione 1:N). Quando un Paziente effettua una prenotazione, viene creato un record nella tabella **APPUNTAMENTI** che si lega a un singolo slot (relazione 1:1), aggiornandone lo stato. Questa separazione è cruciale per prevenire *race condition*: lo slot temporale esiste a prescindere dalla prenotazione, rendendo le transazioni di booking atomicamente sicure. La tabella **APPUNTAMENTI** centralizza inoltre il flusso operativo della visita, tracciando non solo lo stato, ma anche le note del paziente, le **note_medico** e la gestione temporale del check-in in-app (**richiesta_checkin** e **checkin_scadenza**).

Infine, le entità evidenziate nel diagramma illustrano l'implementazione del "Dossier Sanitario Condiviso". La tabella **REFERTI** chiude il ciclo clinico mettendo in relazione il Paziente e il Medico autore. L'accesso trasversale a questa tabella da parte di altri specialisti è strettamente vincolato al campo booleano **consenso_dossier** presente nella

tabella **PAZIENTI**, che traduce a livello di schema fisico i requisiti normativi di *Privacy by Design* e minimizzazione dei dati previsti dal Regolamento Europeo.

Diagramma dei Casi d'Uso (Use Case Diagram)



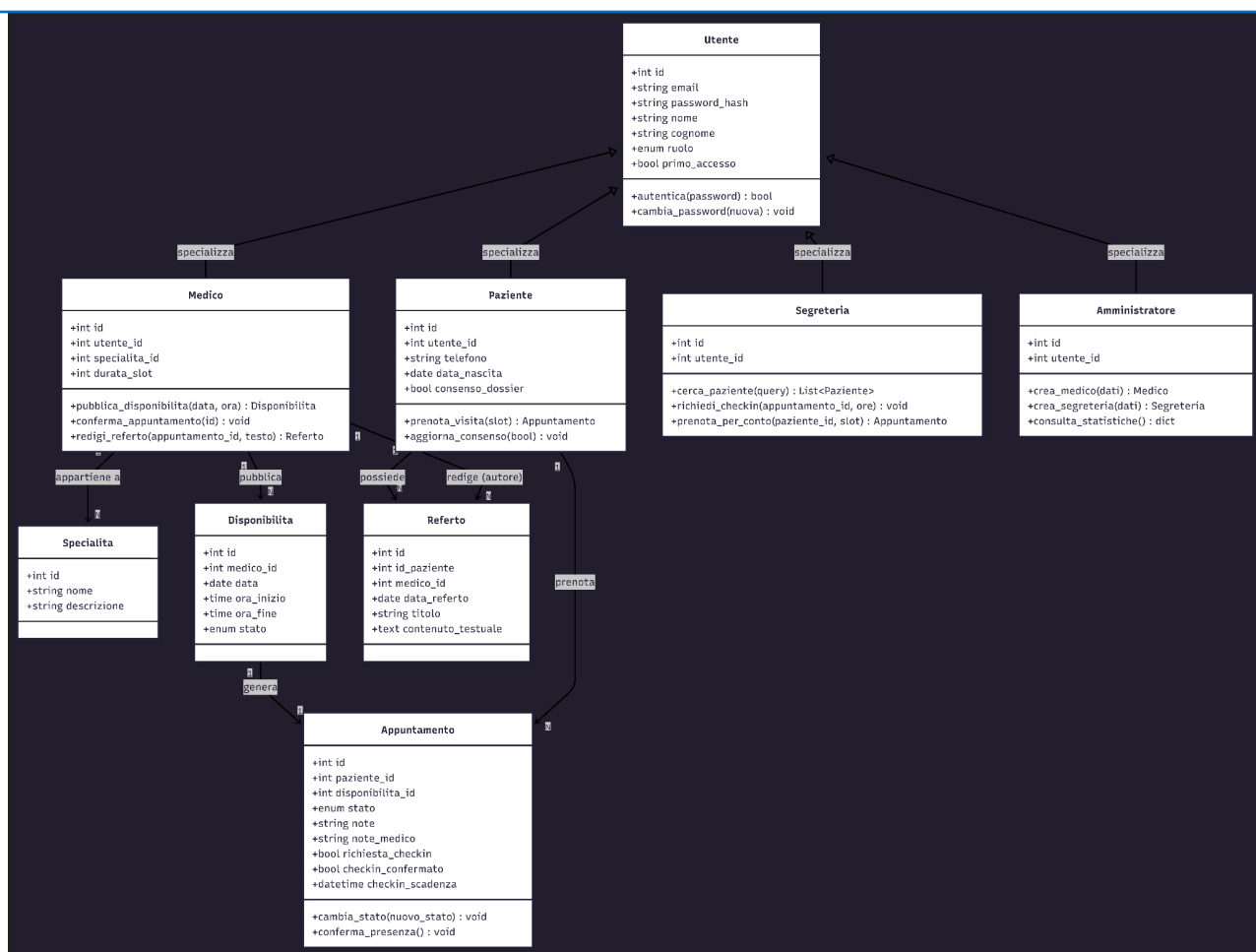
Il diagramma dei casi d'uso evidenzia una netta separazione delle responsabilità tra i quattro attori del sistema, coerente con il principio di separazione dei privilegi (least privilege) adottato a livello di autorizzazione backend.

L'attore **Paziente** rappresenta l'utente finale del servizio ed è caratterizzato da casi d'uso orientati all'autogestione: può prenotare autonomamente una visita attraverso il wizard a quattro passaggi, consultare lo stato dei propri appuntamenti, confermare la propria presenza tramite il sistema di check-in in-app, e soprattutto esercitare un controllo attivo sui propri dati sanitari attraverso il caso d'uso "Gestire il consenso al Dossier Condiviso", che rappresenta l'implementazione concreta del diritto di autodeterminazione informativa discusso nella sezione teorica.

L'attore **Medico** è invece orientato alla gestione clinica: oltre a definire la propria disponibilità oraria e a decidere sull'esito di ciascun appuntamento (conferma, rifiuto, completamento), il medico è l'unico attore abilitato a redigere un referto, azione che ho modellato come inclusione (relazione "include" in notazione UML) del caso d'uso "Consultare il dossier di un paziente", poiché la redazione di un nuovo referto avviene tipicamente nel contesto della consultazione dello storico clinico già disponibile per quel professionista.

Gli attori **Segreteria** e **Amministratore** condividono con gli altri il caso d'uso base di autenticazione, ma si differenziano per l'ambito operativo: la Segreteria svolge un ruolo di intermediazione tra pazienti e medici, potendo effettuare prenotazioni manuali per conto di un paziente e gestire il ciclo di vita del check-in, mentre l'Amministratore opera a un livello di governance dell'intero sistema, occupandosi della gestione anagrafica del personale e delle statistiche aggregate, senza tuttavia avere accesso diretto ai contenuti clinici dei referti, coerentemente con il principio della segregazione dei dati sensibili rispetto ai ruoli non clinici.

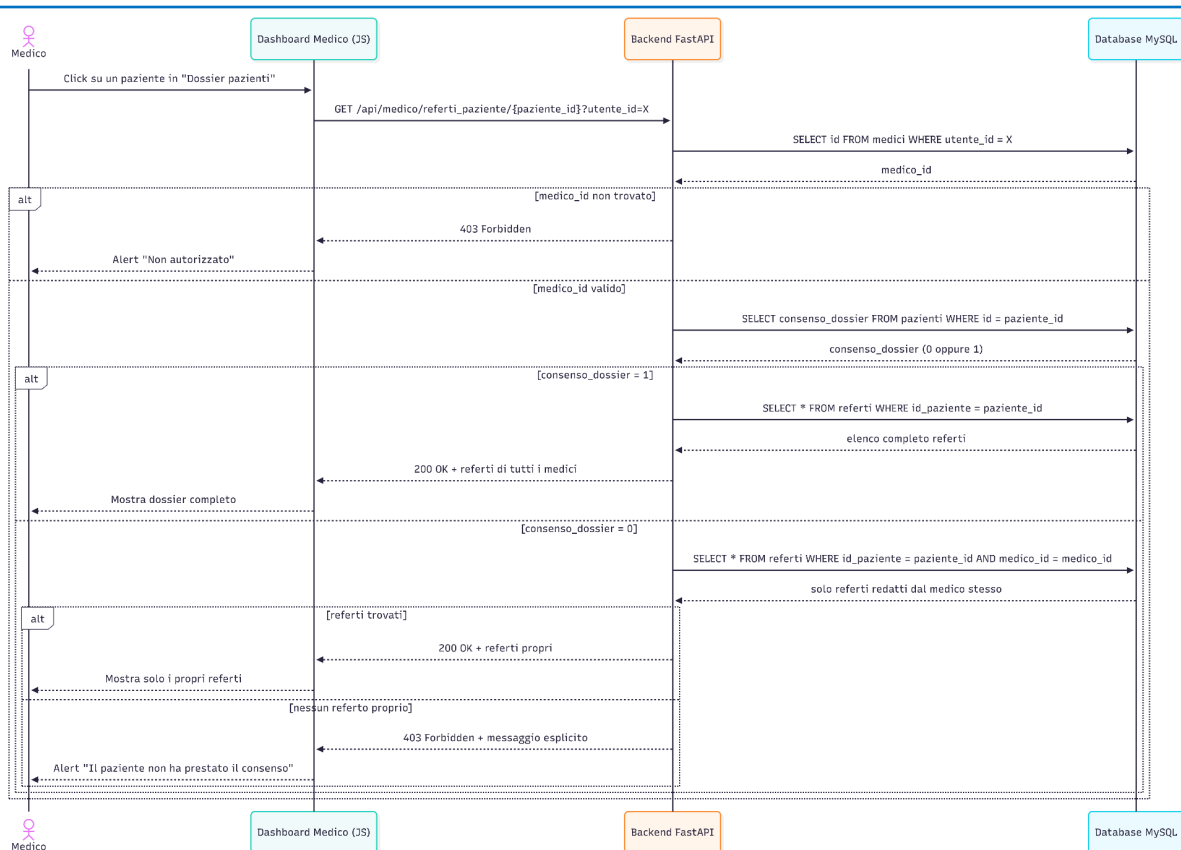
Diagramma delle Classi (Class Diagram)



Il diagramma delle classi rappresenta la struttura logica ad oggetti su cui si fonda il backend dell'applicazione, modellando l'ereditarietà tra l'entità generica Utente e le sue quattro specializzazioni di ruolo: Paziente, Medico, Segreteria e Amministratore. Questa scelta traduce a livello di design il pattern di ereditarietà per tabelle separate già discusso per lo schema del database, evidenziando come ciascun ruolo erediti gli attributi comuni di autenticazione (email, password hash, primo accesso) aggiungendo però responsabilità e attributi propri, quali il flag **consenso_dossier** per il Paziente o la **durata_slot** per il Medico.

Le relazioni evidenziate nel diagramma riflettono fedelmente la cardinalità del modello dati: il Medico pubblica molteplici Disponibilità e redige molteplici Referti in qualità di autore, mentre il Paziente possiede i propri Referti e prenota i propri Appuntamenti. La classe Appuntamento, in particolare, concentra su di sé i metodi di transizione di stato (**cambia_stato**, **conferma_presenza**), coerentemente con il principio di incapsulamento della logica di business all'interno dell'entità che ne è responsabile, anziché disperderla in funzioni esterne prive di legame diretto con i dati che manipolano.

Diagramma di Sequenza (Sequence Diagram)



Il diagramma di sequenza illustra il flusso di interazione più complesso dal punto di vista logico dell'intero sistema: la consultazione dei referti di un paziente da parte di un medico, mediata dal controllo del consenso GDPR. Il diagramma evidenzia due livelli di verifica annidati, rappresentati dai blocchi **alt**: il primo verifica la legittimità del medico richiedente, il secondo, subordinato al superamento del primo, discrimina l'ampiezza dell'accesso concesso in base al valore del flag **consenso_dossier** del paziente.

Nel caso in cui il consenso non sia stato prestato, il diagramma mostra un ulteriore livello di verifica: il sistema interroga comunque il database per verificare se il medico risulti autore di almeno un referto proprio per quel paziente, restituendo tali referti in caso affermativo, oppure un errore 403 esplicito in caso contrario. Questa sequenza dimostra concretamente come il principio di minimizzazione dei dati non sia implementato come un semplice controllo binario, bensì come una logica a più livelli che bilancia la tutela della riservatezza del paziente con la continuità dell'attività clinica del medico.

Documentazione delle API (stile Swagger)

Di seguito documento in dettaglio cinque endpoint rappresentativi dell'architettura REST del sistema, scelti in quanto esemplificativi delle diverse categorie funzionali dell'applicazione: autenticazione, creazione di una risorsa, lettura protetta, aggiornamento di stato con controllo di titolarità, e accesso condizionato a un consenso GDPR.

1. POST /api/login

- **Scopo:** Autentica un utente verificando le credenziali fornite contro l'hash della password memorizzato nel database, e restituisce i dati essenziali del profilo per l'inizializzazione della sessione lato client.

- Request Body (JSON atteso):
- email: paziente@esempio.it
- password: SegretoSicuro123
- Response 200 (Successo):
- id: 6
- nome: Giuseppe
- cognome: Carbone
- ruolo: paziente
- primo_accesso: 0
- Gestione Errori:
 - 401 Unauthorized: Email non registrata o password errata. Il messaggio restituito è volutamente generico ("Credenziali non valide") per prevenire attacchi di enumerazione degli account.
 - 422 Unprocessable Entity: Payload privo dei campi obbligatori o non conforme al modello Pydantic.
- Discussione: La verifica della password avviene tramite `bcrypt.checkpw()`, proteggendo il confronto da timing attack. Il flag `primo_accesso` permette al frontend di gestire il cambio password obbligatorio per i neo-iscritti creati dall'admin.

2. POST /api/prenotazione

- Scopo: Crea un nuovo appuntamento associando un paziente a uno slot di disponibilità libero, aggiornando lo stato dello slot.
- Request Body (JSON atteso):
- utente_id: 6
- disponibilita_id: 14
- note: Prima visita, nessuna allergia nota
- Response 200 (Successo):
- messaggio: Prenotazione effettuata
- Gestione Errori:
 - 404 Not Found: Utente inesistente.
 - 409 Conflict: Slot non più disponibile o già occupato (prevenzione race condition concorrenti).
 - 422 Unprocessable Entity: Campi mancanti.
- Discussione: La transazione garantisce l'atomicità tra l'inserimento dell'appuntamento e l'aggiornamento dello stato dello slot in MySQL prima di eseguire il commit.

3. GET /api/referti

- Scopo: Restituisce l'elenco sintetico dei referti appartenenti al paziente autenticato.
- Parametri: Parametro di query obbligatorio `utente_id`.
- Response 200 (Successo):
- [{ id: 3, data_referto: "2026-07-05", titolo: "Referto visita di Medicina Generale" }]
- Gestione Errori:
 - 403 Forbidden: Utente non autorizzato o ruolo non valido.

- **Discussione:** In osservanza del principio di minimizzazione dei dati (GDPR), la lista restituisce solo i metadati essenziali, rimandando il testo clinico esteso a una chiamata di dettaglio mirata.

4. PATCH /api/paziente/appuntamenti/{appuntamento_id}/cancella

- **Scopo:** Cancella un appuntamento esistente, liberando lo slot di disponibilità associato.
- **Parametri:** Parametro path {appuntamento_id} e query utente_id.
- **Response 200 (Successo):**
- **messaggio:** Appuntamento cancellato
- **Gestione Errori:**
 - **403 Forbidden:** Disallineamento tra il proprietario dell'appuntamento e l'utente richiedente (blocco anti-IDOR).
 - **404 Not Found:** Appuntamento inesistente.

5. GET /api/medico/referti_paziente/{paziente_id}

- **Scopo:** Restituisce lo storico dei referti di un paziente a un medico, applicando le logiche di controllo consenso.
- **Parametri:** Parametro path {paziente_id} e query utente_id del medico.
- **Response 200 (Successo - con consenso):**
- **[{ id: 3, data_referto: "2026-07-05", titolo: "Referto visita", contenuto_testuale: "Esito...", medico_nome: "Mario" }]**
- **Gestione Errori:**
 - **403 Forbidden:** Paziente privo di consenso attivo al Dossier Sanitario Condiviso e medico non titolare diretto della refertazione.

Test funzionale dell'applicazione

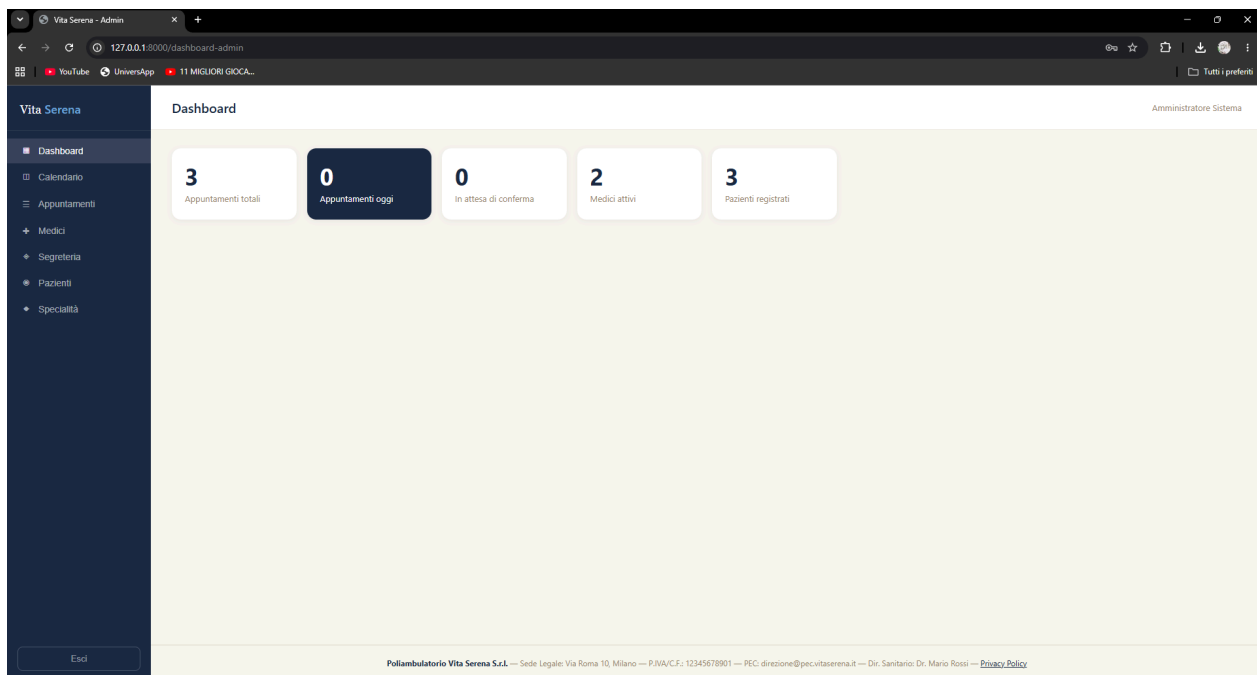
In questa sezione descrivo in maniera dettagliata il comportamento dell'applicazione dal punto di vista dell'utente finale, illustrando i flussi di navigazione, le interazioni disponibili e i meccanismi di feedback implementati in ciascuna delle quattro dashboard del sistema.

La descrizione che segue è stata verificata attraverso test manuali ripetuti su ciascun ruolo, effettuati in ambiente locale con database MySQL popolato da dati di prova realistici.

Dashboard Amministratore

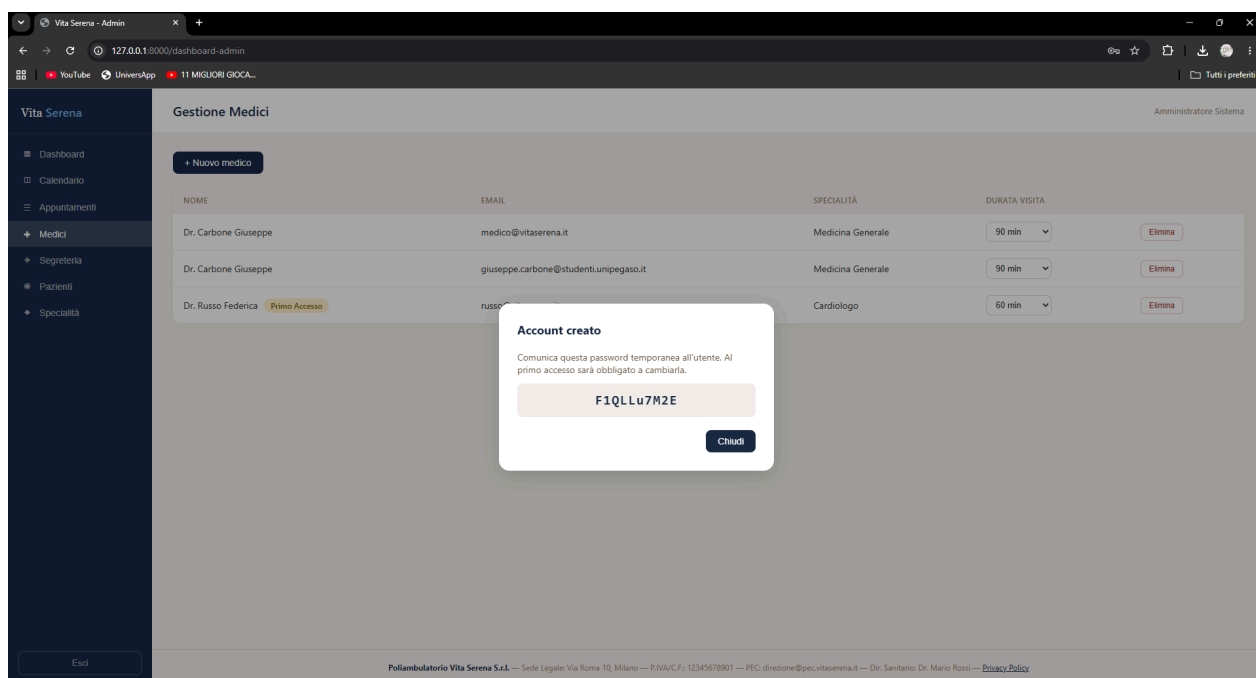
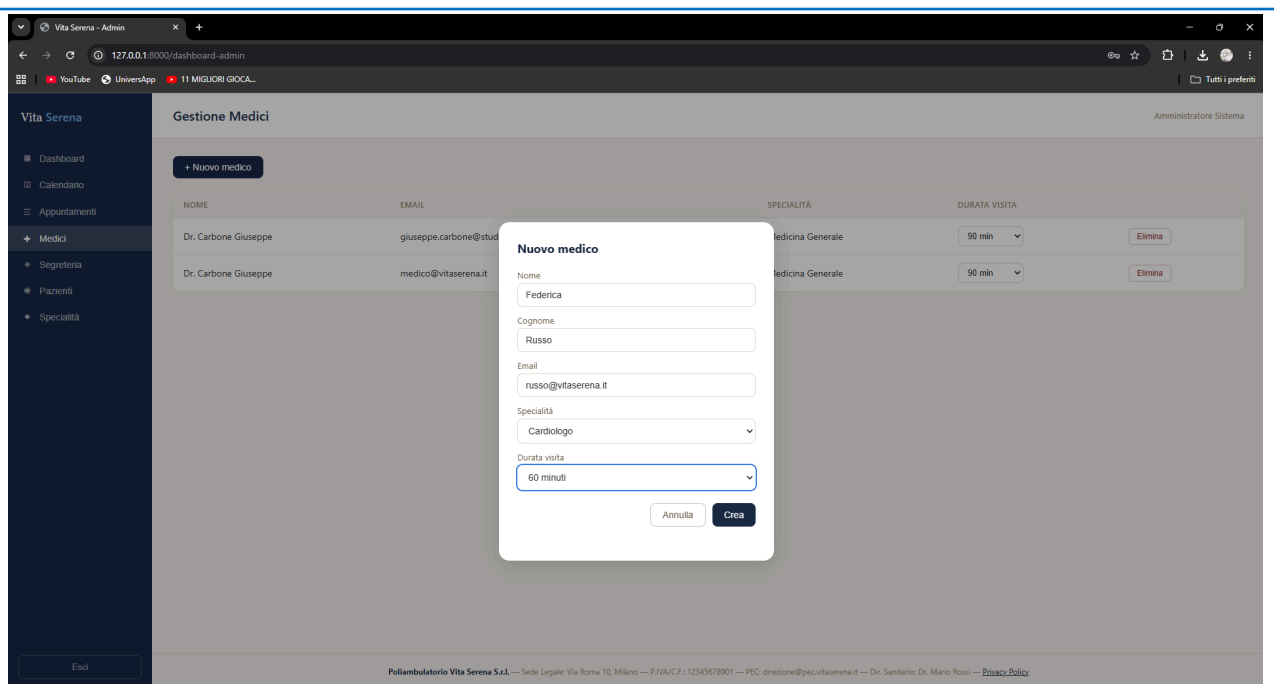
All'accesso, l'amministratore visualizza la sezione "Dashboard", contenente cinque riquadri statistici che riportano rispettivamente il numero totale di appuntamenti registrati nel sistema, gli appuntamenti previsti per la giornata corrente, il numero di prenotazioni in

stato di attesa di conferma, il numero di medici attivi e il numero di pazienti registrati. Questi valori vengono recuperati tramite una singola chiamata all'endpoint `/api/admin/stats` all'apertura della pagina.



Spostandosi sulla sezione "Gestione Medici" tramite il menu laterale, l'amministratore visualizza una tabella con l'elenco completo dei medici registrati, comprensiva di nome, email, specialità di appartenenza e un menu a tendina che consente di modificare in tempo reale la durata dello slot di visita assegnata a ciascun professionista, tra i valori disponibili di 15, 20, 30, 45, 60 o 90 minuti.

Cliccando sul pulsante "Nuovo medico", si apre una finestra modale contenente un modulo per l'inserimento dei dati anagrafici e della specialità: al salvataggio, il sistema genera automaticamente una password temporanea, mostrata in una seconda finestra modale, da comunicare manualmente al nuovo medico affinché la utilizzi per il primo accesso.

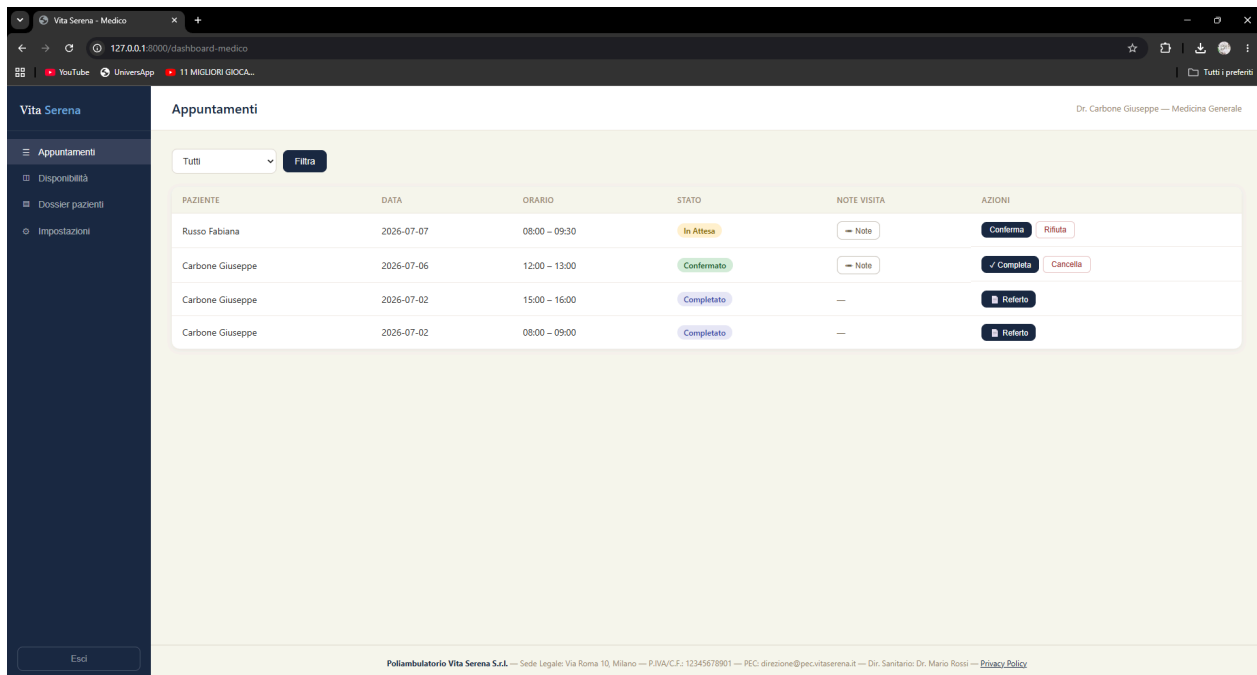


La sezione "Specialità" consente analogamente la gestione del catalogo delle specializzazioni mediche disponibili, mentre la sezione "Pazienti" offre una vista di sola lettura dell'anagrafica dei pazienti registrati, deliberatamente priva di accesso ai dati clinici, in coerenza con il principio di segregazione dei dati sensibili rispetto ai ruoli non clinici discusso nella sezione relativa alla sicurezza.

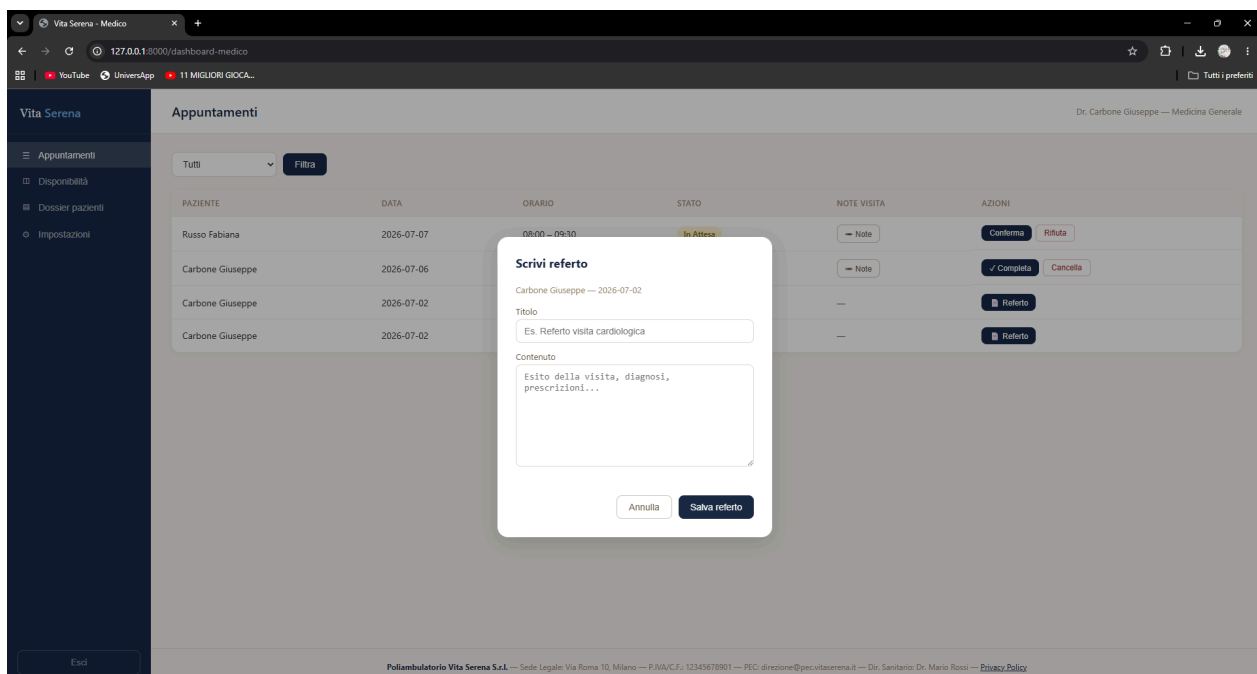
Dashboard Medico

Il medico, all'accesso, visualizza per impostazione predefinita la sezione "Appuntamenti", strutturata come una tabella filtrabile per stato (in attesa, confermati, cancellati, completati) tramite un menu a tendina posizionato nella parte superiore della pagina. Ogni riga della tabella riporta il nome del paziente, la data e l'orario dell'appuntamento, un'etichetta colorata (badge) che ne indica lo stato corrente, e una colonna di azioni contestuali che varia dinamicamente in base allo stato: per un appuntamento in attesa

sono disponibili i pulsanti "Conferma" e "Rifiuta", mentre per un appuntamento confermato sono disponibili i pulsanti "Completa" e "Cancella".

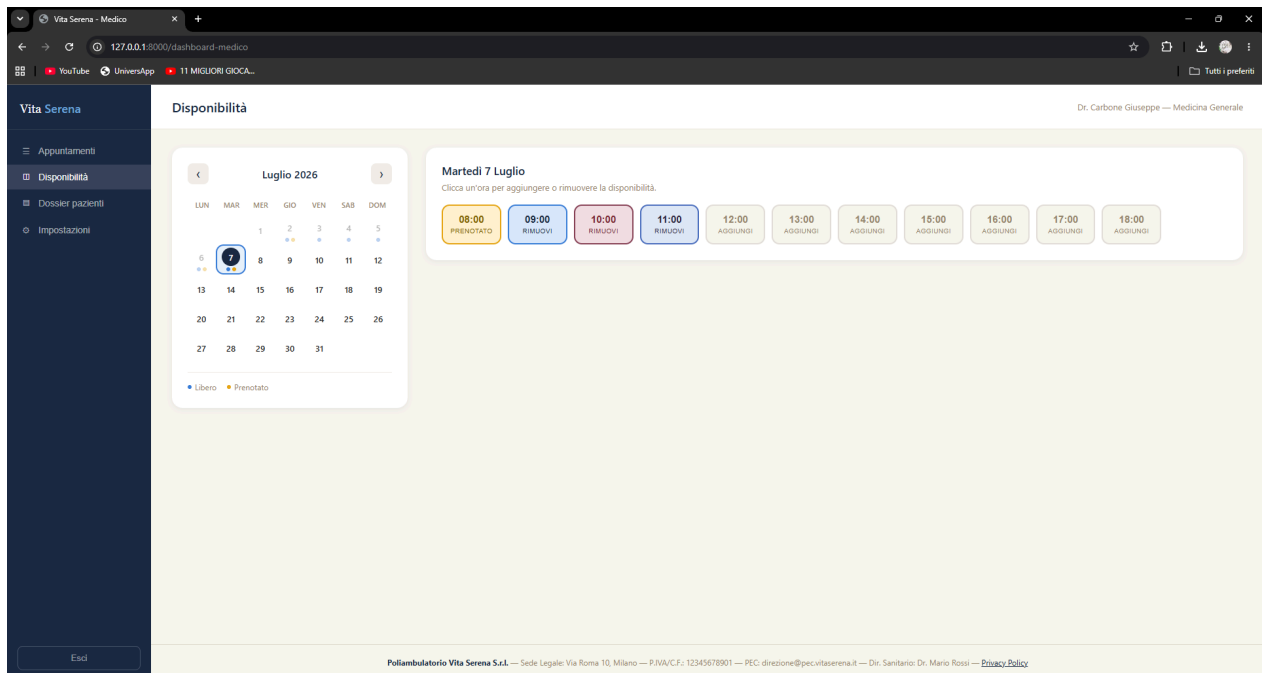


Una volta portato un appuntamento allo stato "completato", compare un ulteriore pulsante "Referto" che apre una finestra modale contenente un modulo con i campi "Titolo" e "Contenuto", destinato alla redazione del referto clinico relativo alla visita appena conclusa. Il sistema impedisce la redazione di un referto per appuntamenti non ancora completati, restituendo un messaggio di errore esplicito qualora tale vincolo venga aggirato tramite una richiesta diretta all'API.

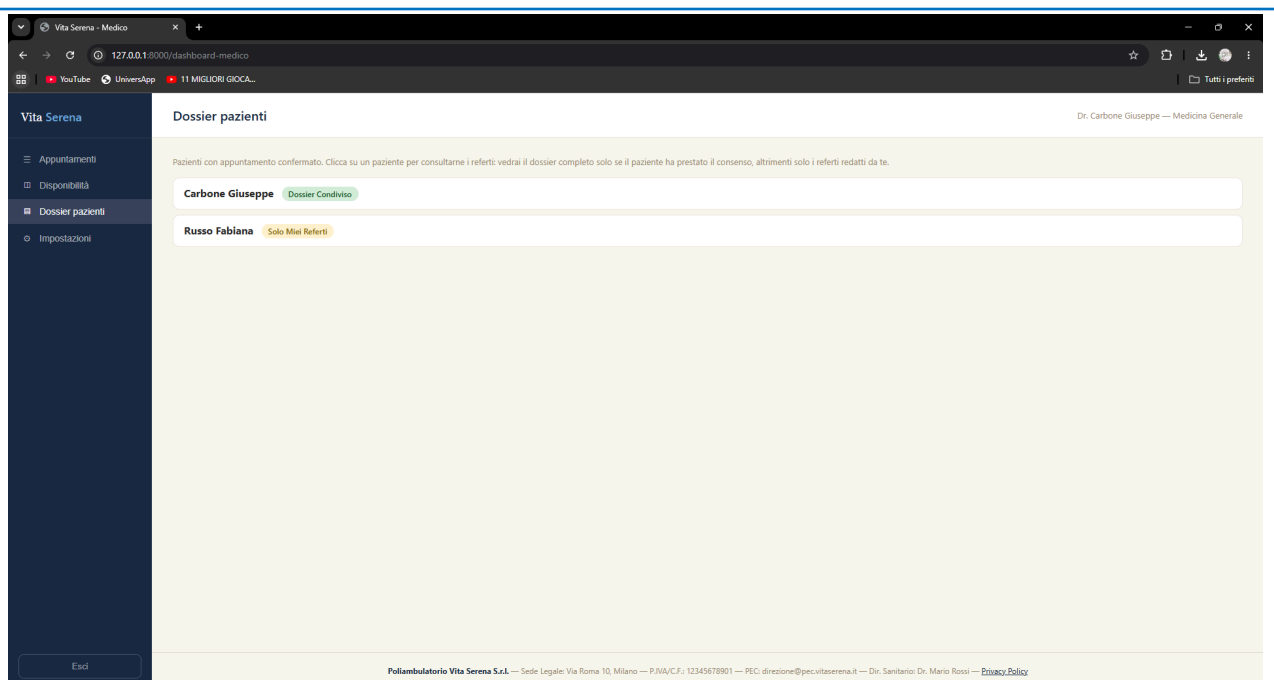


La sezione "Disponibilità" presenta un calendario mensile con indicatori visivi puntiformi (dot) di colore differente per segnalare, in corrispondenza di ciascun giorno, la presenza di slot liberi (colore blu) o già prenotati (colore giallo). Selezionando un giorno del calendario, il pannello laterale mostra una griglia di bottoni orari, uno per ciascuna fascia

disponibile nell'arco della giornata lavorativa: un bottone vuoto consente di aggiungere un nuovo slot con un semplice clic, un bottone già occupato da uno slot libero ne consente la rimozione, mentre un bottone corrispondente a uno slot già prenotato risulta disabilitato e non modificabile.

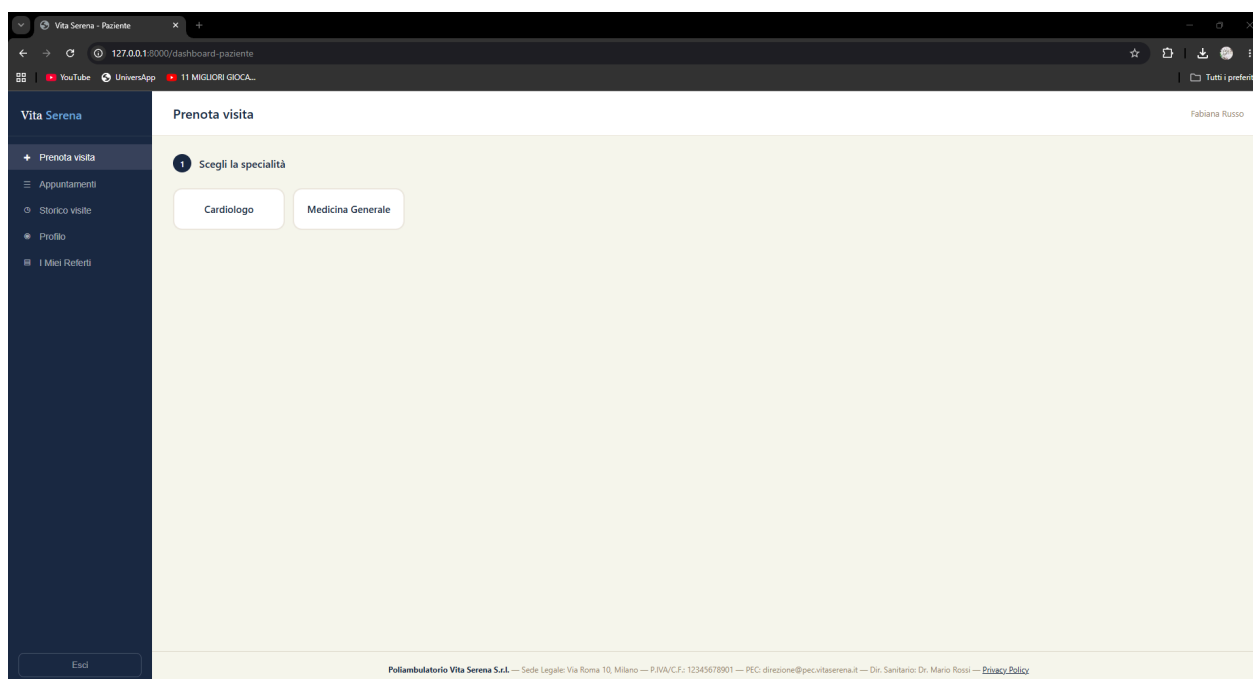


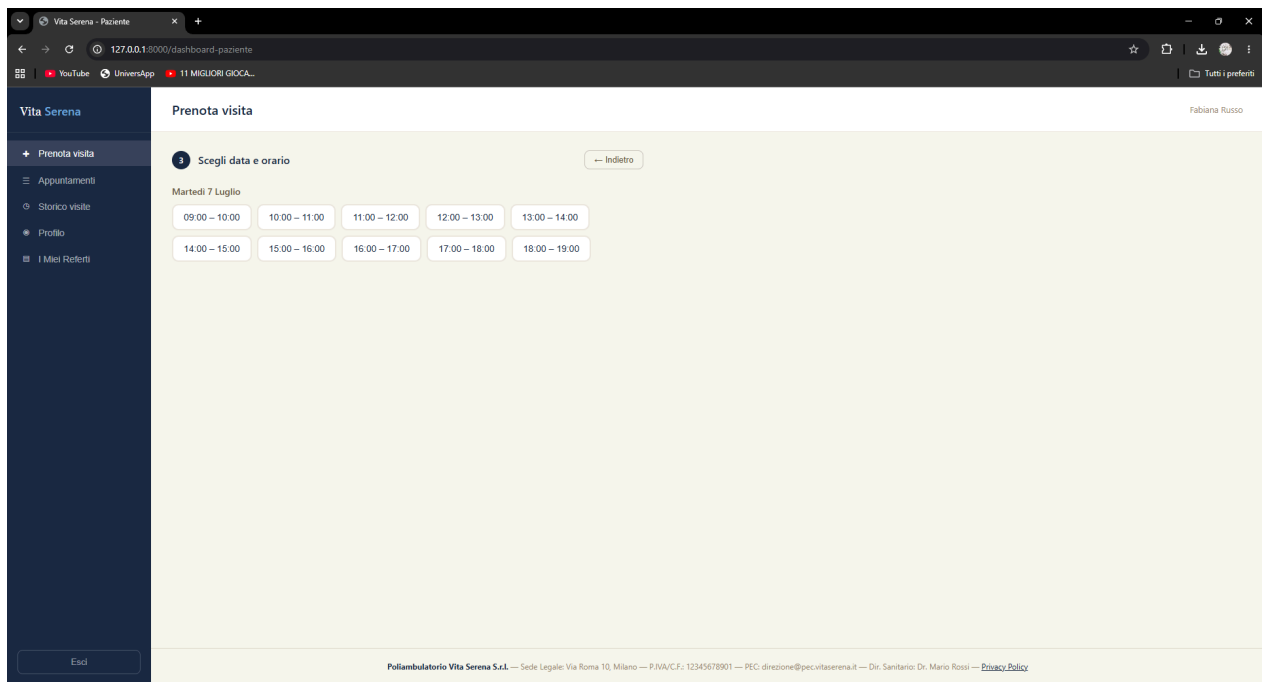
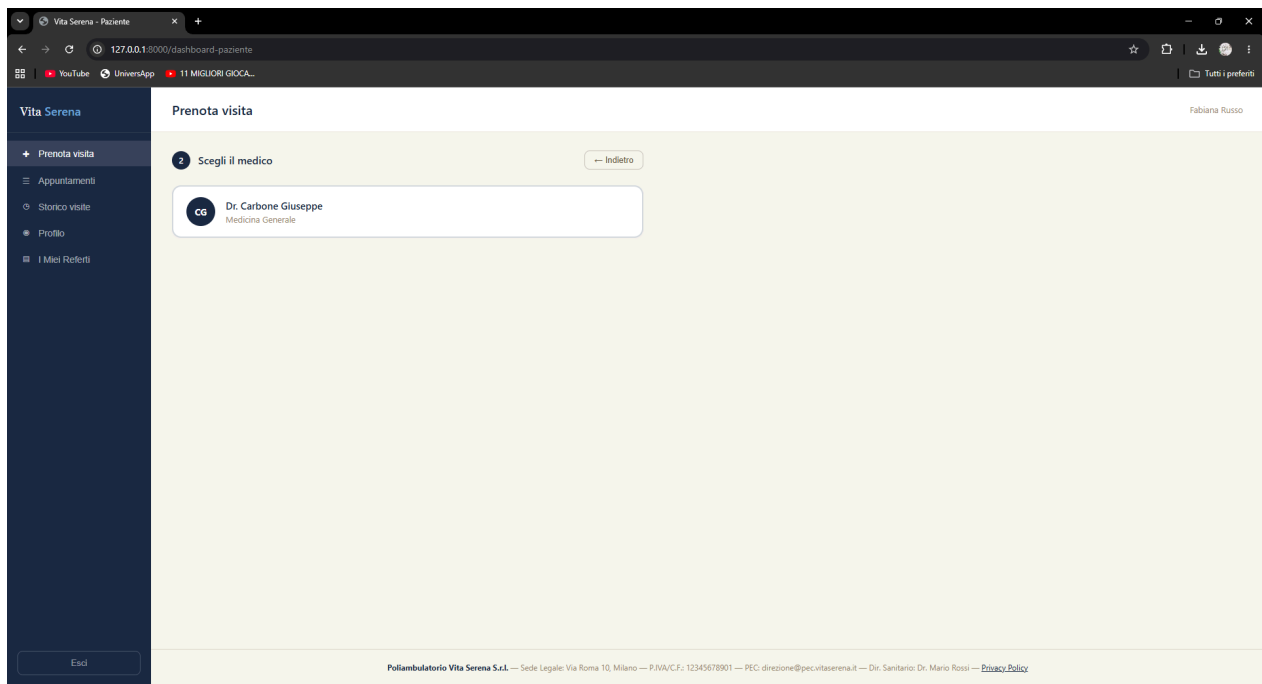
La nuova sezione "Dossier pazienti" elenca i pazienti con almeno un appuntamento confermato presso il medico, ciascuno accompagnato da un'etichetta che indica se il paziente ha prestato il consenso al Dossier Sanitario Condiviso ("Dossier condiviso") oppure se l'accesso è limitato ai soli referti redatti dal medico stesso ("Solo miei referti"). Cliccando su un paziente, il sistema tenta di recuperare lo storico dei referti: qualora il consenso non sia stato prestato e il medico non risulti autore di alcun referto pregresso per quel paziente, l'applicazione mostra un avviso a schermo con il messaggio "Il paziente non ha prestato il consenso per la visualizzazione dei referti pregressi", impedendo la visualizzazione di dati clinici non autorizzati.



Dashboard Paziente

Il paziente accede per impostazione predefinita al wizard di prenotazione, articolato in quattro passaggi progressivi: la selezione della specialità desiderata, la scelta del medico tra quelli disponibili per la specialità selezionata, la scelta dello slot orario tra quelli liberi raggruppati per data, e infine una schermata di riepilogo con la possibilità di aggiungere una nota facoltativa prima di confermare la prenotazione. Ogni passaggio del wizard include un pulsante "Indietro" che consente di tornare allo step precedente senza perdere le selezioni già effettuate.





The screenshot shows the 'Prenota visita' page in the Vita Serena patient dashboard. The page is titled 'Prenota visita' and features a sidebar with navigation options: 'Prenota visita', 'Appuntamenti', 'Storico visite', 'Profilo', and 'I miei Referti'. The main content area is titled '4 Conferma prenotazione' and includes a 'Indietro' button. The appointment details are as follows:

Specialità	Medicina Generale
Medico	Dr. Carbone Giuseppe
Data	martedì 7 luglio 2026
Orario	09:00 - 10:00

Below the details is a text input field for 'Note (opzionale)' with the placeholder text 'Es. allergie, sintomi...'. At the bottom of the form is a 'Conferma prenotazione' button. The footer of the page contains the following text: 'Pollambulatorio Vita Serena S.r.l. — Sede Legale: Via Roma 10, Milano — P.IVA/C.F.: 12345678901 — PEC: direzione@pec.vitaserena.it — Dir. Sanitario: Dr. Mario Rossi — [Privacy Policy](#)'.

Nella sezione "Appuntamenti" il paziente visualizza l'elenco dei propri appuntamenti futuri, filtrabili per specialità, ciascuno rappresentato da una scheda (card) che ne riporta i dettagli e un'etichetta di stato colorata; è presente inoltre un pulsante "Cancella" che, previa conferma tramite finestra di dialogo del browser, annulla l'appuntamento e libera nuovamente lo slot corrispondente.

The screenshot shows the 'I miei appuntamenti' page in the Vita Serena patient dashboard. The page is titled 'I miei appuntamenti' and features a sidebar with navigation options: 'Prenota visita', 'Appuntamenti', 'Storico visite', 'Profilo', and 'I miei Referti'. The main content area includes a filter section with a dropdown menu set to 'Tutte le specialità' and a 'Filtra' button. Below the filter are two appointment cards:

Specialità	Medico	Data	Orario	Stato	Azioni
Medicina Generale	Dr. Carbone Giuseppe	Martedì 7 Luglio 2026	09:00 - 10:00	In Attesa	Cancella
Medicina Generale	Dr. Carbone Giuseppe	Martedì 7 Luglio 2026	08:00 - 09:30	Confermato	Cancella

The footer of the page contains the following text: 'Pollambulatorio Vita Serena S.r.l. — Sede Legale: Via Roma 10, Milano — P.IVA/C.F.: 12345678901 — PEC: direzione@pec.vitaserena.it — Dir. Sanitario: Dr. Mario Rossi — [Privacy Policy](#)'.

La sezione "I miei referti" mostra l'elenco dei referti disponibili, ciascuno con titolo e data; cliccando su un referto si apre una finestra modale con il contenuto testuale completo. Nella sezione "Profilo", oltre alla modifica dei dati anagrafici, è presente il selettore per la gestione del consenso al Dossier Sanitario Condiviso: attivando la casella di controllo, il

sistema invia immediatamente una richiesta all'endpoint dedicato e mostra un messaggio di conferma testuale, senza necessità di un ulteriore salvataggio esplicito.

The screenshot shows the 'Il mio profilo' page for Fabiana Russo. The left sidebar contains navigation links: 'Prenota visita', 'Appuntamenti', 'Storico visite', 'Profilo' (selected), and 'I Miei Referti'. The main content area displays the patient's name and email, followed by input fields for 'Telefono' (3518142813) and 'Data di nascita' (15/06/2006), with a 'Salva modifiche' button. Below this is a section for 'Dossier Sanitario Condiviso' with a checkbox for consent. The footer includes the company name 'Pollambulatorio Vita Serena S.r.l.' and contact information.

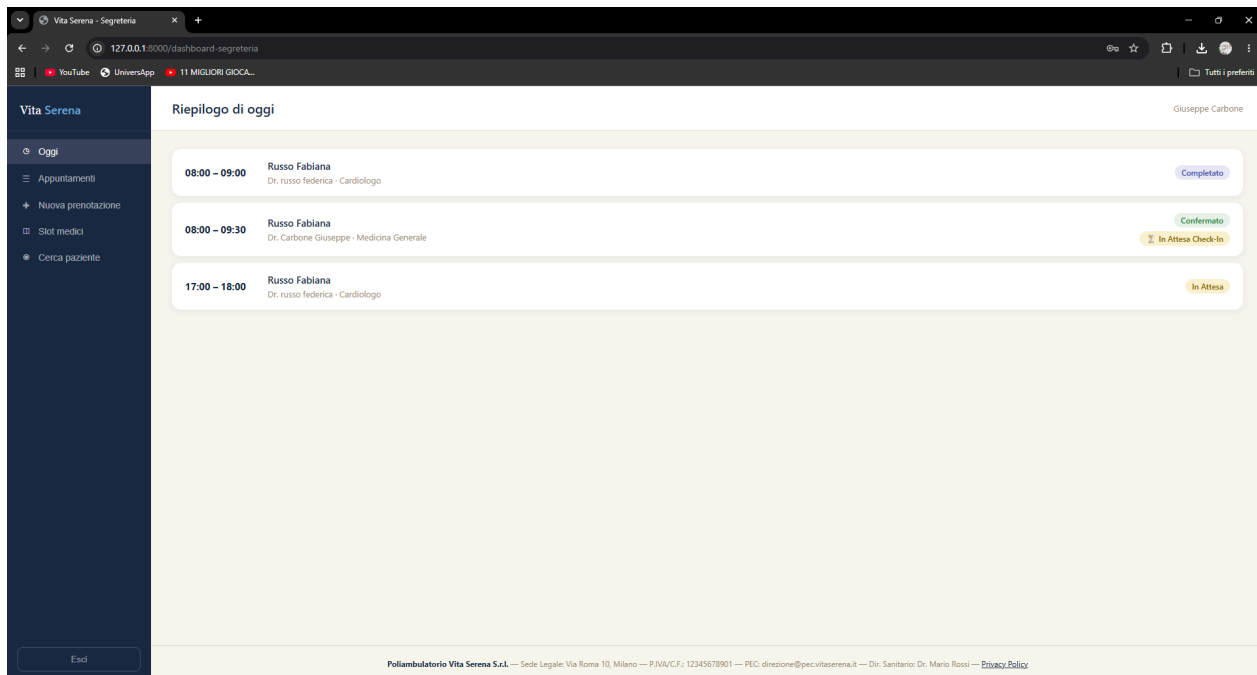
Infine, un banner giallo posizionato nella parte superiore di ogni sezione della dashboard notifica al paziente eventuali richieste di conferma della presenza (check-in) pendenti, riportando la scadenza entro cui è necessario rispondere e un pulsante "Confermo la presenza" per completare l'operazione.

The screenshot shows the 'Prenota visita' page. A yellow banner at the top indicates a pending 'Conferma presenza' for a visit with Dr. Carbone Giuseppe on Tuesday, July 7th at 08:00, with a 'Confermo la presenza' button. Below the banner, a section titled 'Scegli la specialità' (Choose the specialty) shows two buttons: 'Cardiologo' and 'Medicina Generale'. The left sidebar is identical to the previous screenshot. The footer is also the same.

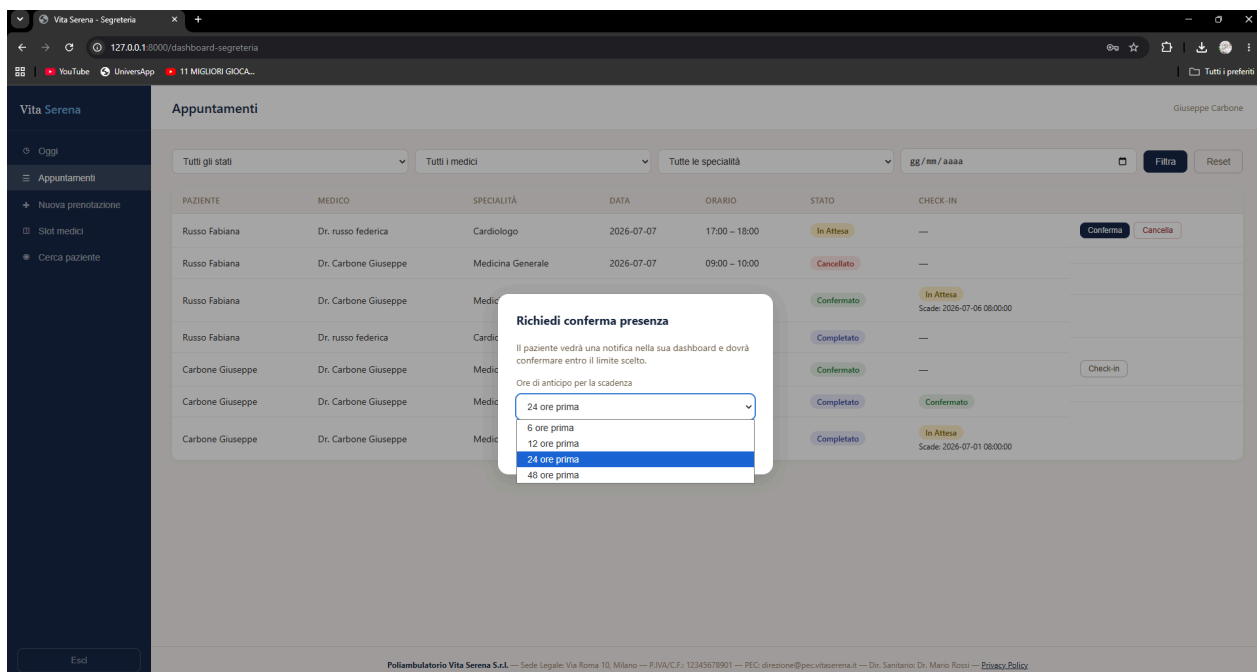
Dashboard Segreteria

La segreteria accede per impostazione predefinita alla sezione "Oggi", che presenta l'elenco degli appuntamenti previsti per la giornata corrente in formato tabellare

compatto, con indicazione dell'orario, del paziente, del medico e della specialità, oltre a un'etichetta relativa allo stato del check-in, se richiesto.



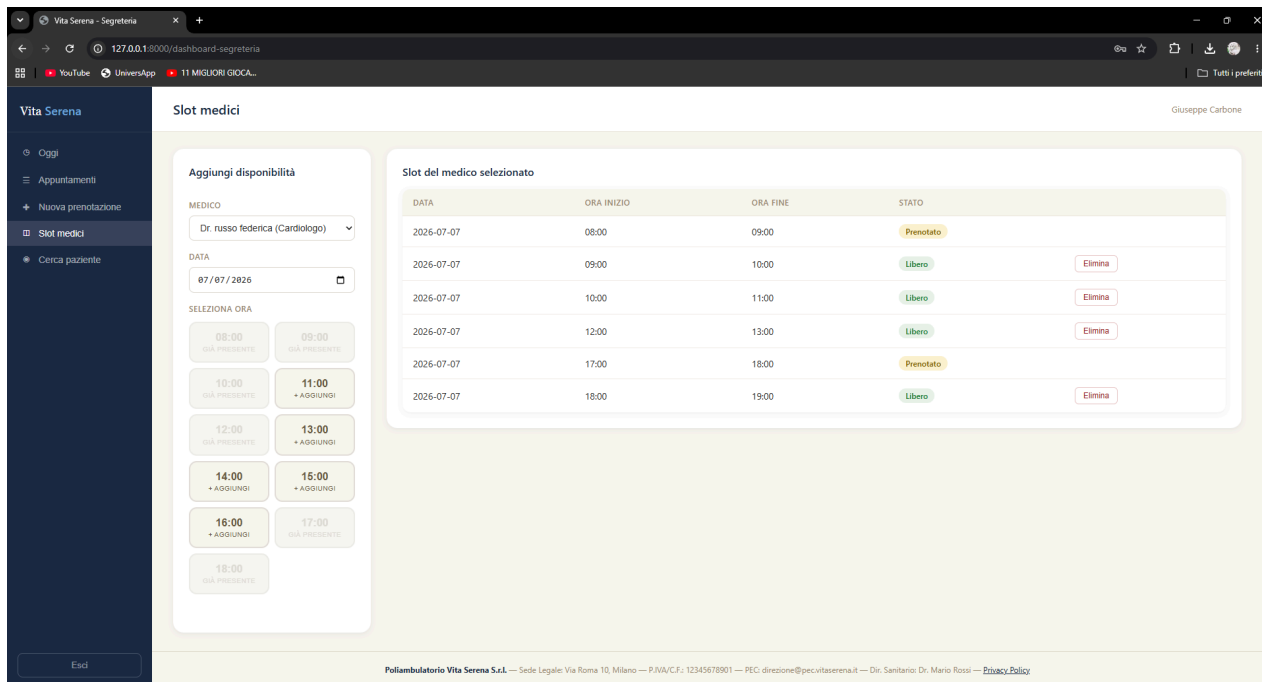
Nella sezione "Appuntamenti", la segreteria dispone di un set di filtri combinabili (stato, medico, specialità, data specifica) per ricercare rapidamente un sottoinsieme di prenotazioni, con un pulsante "Reset" che azzerava tutti i filtri applicati. Da questa vista è possibile anche avviare la richiesta di check-in per un appuntamento confermato, tramite un pulsante che apre una finestra modale dove selezionare il numero di ore di anticipo desiderato (6, 12, 24 o 48 ore) prima della scadenza automatica della richiesta.



La sezione "Nuova prenotazione" replica per la segreteria un wizard analogo a quello del paziente, con l'aggiunta di un passaggio iniziale di ricerca anagrafica del paziente per

cognome, nome o email, necessario poiché la prenotazione viene effettuata per conto di un utente terzo.

La sezione "Slot medici" consente infine di gestire le disponibilità di qualsiasi medico della struttura, selezionandolo da un menu a tendina e utilizzando la medesima interfaccia a bottoni orari già descritta per la dashboard del medico.



Ulteriori snippet di codice commentati (Appendice)

Setup della connessione al database con PyMySQL (**main.py**)

Python

```
def get_connection():  
    return pymysql.connect(  
        host="localhost",  
        user="root",  
        password="",  
        database="vitaserena",  
        cursorclass=pymysql.cursors.DictCursor  
    )
```

Questa funzione, per quanto sinteticamente breve, racchiude una scelta progettuale rilevante ai fini della leggibilità dell'intero backend. La riga `cursorclass=pymysql.cursors.DictCursor` configura il driver affinché ogni riga restituita da una query venga rappresentata come un dizionario Python, le cui chiavi corrispondono ai nomi delle colonne selezionate, anziché come una tupla posizionale, comportamento predefinito del driver PyMySQL. Ho adottato questa configurazione fin

dalle prime fasi del progetto poiché elimina la necessità di accedere ai risultati tramite indici numerici posizionali (ad esempio `riga[0]`, `riga[1]`), pratica fragile e soggetta a errori qualora l'ordine delle colonne nella clausola `SELECT` venga modificato in un secondo momento. L'accesso tramite chiave semantica (`riga["nome"]`, `riga["cognome"]`) rende inoltre il codice autoesplicativo e direttamente compatibile con la serializzazione JSON richiesta da FastAPI, poiché un dizionario Python viene convertito automaticamente in un oggetto JSON, mentre una tupla richiederebbe una trasformazione esplicita e manuale. La funzione viene invocata singolarmente all'inizio di ciascun endpoint, secondo un pattern che rinuncia deliberatamente a un pool di connessioni condiviso, in favore di una gestione esplicita e localizzata del ciclo di vita di ciascuna connessione, aperta e chiusa entro i confini del blocco `try/finally` di ogni singola funzione.

Hashing della password e verifica delle credenziali con `bcrypt` (`main.py`)

Python

```
password_hash = bcrypt.hashpw(
    dati.password.encode("utf-8"),
    bcrypt.gensalt()
).decode()

# In fase di login:
password_ok = bcrypt.checkpw(
    credenziali.password.encode("utf-8"),
    utente["password"].encode("utf-8")
)
```

Questo snippet illustra l'implementazione del meccanismo di autenticazione crittograficamente sicuro adottato per la protezione delle password degli utenti. La funzione `bcrypt.gensalt()` genera un valore casuale (salt) univoco per ogni singola password, che viene incorporato nell'hash finale prodotto da `bcrypt.hashpw()`: questo accorgimento garantisce che due utenti con la medesima password in chiaro producano due hash memorizzati completamente differenti nel database, vanificando l'efficacia di attacchi basati su tabelle precalcolate (rainbow table). La chiamata `.encode("utf-8")` è necessaria poiché la libreria `bcrypt` opera su sequenze di byte anziché su stringhe Python native, mentre la successiva `.decode()` riconverte l'hash prodotto in una stringa, formato più agevole per la persistenza nella colonna `password` di tipo `VARCHAR`. In fase di verifica delle credenziali, la funzione `bcrypt.checkpw()` non decodifica l'hash memorizzato per confrontarlo con la password fornita, operazione impossibile data la natura unidirezionale della funzione di hash, bensì ricalcola l'hash della password inserita dall'utente utilizzando il medesimo salt già incorporato nell'hash memorizzato, e ne confronta il risultato byte per byte. Questo approccio, oltre a essere crittograficamente

robusto, è implementato internamente dalla libreria in un tempo costante rispetto al contenuto confrontato, contromisura contro attacchi di tipo timing attack che tentassero di dedurre la password corretta misurando le minime variazioni nel tempo di risposta del server.

Wrapper JavaScript per le chiamate Fetch con gestione centralizzata degli errori (**utils.js** / dashboard JS)

JavaScript

```
async function api(metodo, url, body) {
  const opzioni = {
    method: metodo,
    headers: { 'Content-Type': 'application/json' }
  };
  if (body) {
    opzioni.body = JSON.stringify(body);
  }
  const risposta = await fetch(url, opzioni);
  const dati = await risposta.json();
  return { ok: risposta.ok, dati };
}
```

Questa funzione, replicata con lievi variazioni in ciascuno dei quattro file JavaScript principali del frontend, rappresenta un pattern di astrazione (wrapper) che ho introdotto per evitare la ripetizione della medesima logica di configurazione della richiesta HTTP in ogni singola chiamata all'API. La funzione accetta tre parametri: il metodo HTTP da utilizzare, l'URL dell'endpoint di destinazione, e un oggetto opzionale rappresentante il corpo della richiesta, che viene serializzato in formato JSON tramite **JSON.stringify()** solamente qualora sia effettivamente presente, evitando di inviare un corpo vuoto o non necessario nelle richieste di tipo GET o DELETE prive di payload. L'aspetto più rilevante dal punto di vista della gestione degli errori risiede nel valore di ritorno della funzione: anziché propagare un'eccezione in caso di risposta con codice di errore HTTP, come farebbe di default l'API Fetch nativa (che non considera un codice 404 o 403 come un errore da intercettare tramite blocco **catch**), la funzione restituisce sempre un oggetto contenente la proprietà booleana **ok**, ottenuta direttamente dalla proprietà omonima dell'oggetto **Response**, e la proprietà **dati**, contenente il corpo della risposta già deserializzato. Questo pattern consente a ogni funzione chiamante di adottare un'unica struttura di controllo uniforme, del tipo **const { ok, dati } = await api(...); if (!ok) { /* gestione errore */ }**, riducendo la duplicazione di blocchi **try/catch** sparsi nel codice e centralizzando in un unico punto la logica di interpretazione della risposta HTTP, un principio di ingegneria del software noto come "Don't Repeat Yourself" (DRY).

Gestione della serializzazione JSON per i tipi data/timedelta di MySQL (main.py)

Uno dei problemi tecnici più sottili incontrati nello sviluppo è stata la gestione dei tipi di dato restituiti dal driver MySQL PyMySQL.

Quando si esegue una query che include colonne di tipo DATE, DATETIME o TIME, PyMySQL non restituisce stringhe ma oggetti Python nativi: rispettivamente datetime.date, datetime.datetime e datetime.timedelta.

Il modulo json standard di Python e di conseguenza FastAPI non sa come serializzare questi tipi, causando un errore a runtime.

La soluzione adottata è stata la funzione helper serializza_righe(), applicata a tutti gli endpoint che restituiscono righe contenenti tali colonne:

Python

```
def serializza_righe(rows):
    for r in rows:
        for k, v in r.items():
            if isinstance(v, (datetime.date, datetime.datetime)):
                r[k] = v.isoformat()
            elif isinstance(v, datetime.timedelta):
                total = int(v.total_seconds())
                r[k] = f"{total // 3600:02d}:{(total % 3600) //
60:02d}"
    return rows
```

La funzione itera su tutte le righe e su tutti i campi: se il valore è una data o datetime, lo converte in stringa ISO 8601 (es. "2026-07-02"); se è un timedelta (formato restituito da MySQL per le colonne TIME), lo converte in una stringa HH:MM calcolando ore e minuti dai secondi totali.

Questo approccio è preferibile all'uso di un encoder JSON personalizzato perché è esplicito, localizzato e non richiede configurazione globale del framework.

Ho inoltre scelto di applicare la funzione singolarmente a ogni endpoint che la necessita, anziché intercettare la conversione a livello centrale (ad esempio tramite un JSONEncoder custom passato a FastAPI), per mantenere visibile ed esplicito, endpoint per endpoint, quali risposte contengono campi temporali da trattare una scelta coerente con l'approccio di gestione esplicita delle risorse già adottato per la connessione al database, preferendo la chiarezza locale a un'astrazione implicita e distribuita nel framework.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

Campi di applicazione

L'applicazione "Vita Serena" è stata progettata specificamente per il settore sanitario privato, ma i suoi principi architettureali sono applicabili in numerosi altri contesti.

Applicazioni dirette nel settore sanitario:

- Poliambulatori privati e studi medici associati
- Cliniche specialistiche (odontoiatria, fisioterapia, psicologia)
- Centri diagnostici per la prenotazione di esami
- Studi medici di medicina generale per la gestione delle visite

Impatto quantificato dell'introduzione del sistema

Per stimare l'impatto economico e organizzativo dell'adozione della piattaforma, ho preso come riferimento un poliambulatorio di dimensioni medie con 5 medici attivi, una segreteria composta da 2 addetti, e un volume medio di 40 prenotazioni giornaliere gestite prima manualmente per telefono.

- **Tempo risparmiato in segreteria:** nella gestione manuale, ogni prenotazione telefonica richiede in media 4-5 minuti tra ricerca della disponibilità, verifica dei dati del paziente e conferma. Con la prenotazione autonoma online, stimo che almeno il 60% delle prenotazioni (circa 24 al giorno) venga effettuato direttamente dal paziente senza intervento della segreteria, con un risparmio stimato di circa **1 ora e 40 minuti al giorno per addetto**, equivalenti a oltre **35 ore mensili** di tempo liberato da attività ripetitive e destinabile a mansioni di maggior valore (accoglienza, gestione di casi complessi, supporto amministrativo).
- **Riduzione dei costi legati agli errori di doppia prenotazione:** nella gestione cartacea, l'incidenza tipica di doppie prenotazioni sullo stesso slot si attesta, per esperienza empirica di settore, tra il 3% e il 5% del totale. Su un volume di 40 prenotazioni giornaliere, ciò equivale a 1-2 conflitti al giorno, ciascuno dei quali richiede in media 10-15 minuti di gestione telefonica per la ricollocazione del paziente. Il controllo di disponibilità in tempo reale a livello di database elimina strutturalmente questo problema, con un risparmio stimato di circa **5-8 ore mensili** di gestione dei conflitti.
- **Riduzione del tasso di mancata presentazione (no-show):** il tasso medio di no-show in ambito ambulatoriale privato, secondo dati di settore, si attesta tra il 10% e il 20% degli appuntamenti fissati. Il sistema di check-in in-app, sollecitando attivamente la conferma di presenza con un anticipo configurabile, può ragionevolmente ridurre tale tasso di 3-5 punti percentuali. Su un volume di 40 visite giornaliere con una tariffa media di 50 euro a prestazione, una riduzione del 4% del no-show equivale a circa **1,6 visite recuperate al giorno**, corrispondenti a un maggiore introito stimato di circa **80 euro al giorno**, ovvero **oltre 1.700 euro**

mensili su base di 22 giorni lavorativi.

- **Efficientamento della manodopera medica:** la possibilità di configurare la durata dello slot per ciascun medico in base alla propria specialità consente di ottimizzare il numero di visite erogabili nell'arco della giornata lavorativa. Un medico di medicina generale che riduce il proprio slot standard da 20 a 15 minuti, laddove clinicamente appropriato, guadagna la possibilità di erogare fino a **2-3 visite aggiuntive** in una giornata lavorativa di 8 ore, con un incremento potenziale della produzione ambulatoriale stimabile attorno al **15-20%** per i professionisti che adottano tale ottimizzazione.
- **Tracciabilità e riduzione del rischio di contenzioso:** la registrazione sistematica di ogni azione con marcatura temporale (creazione appuntamento, cambio di stato, redazione referto) riduce il rischio di contestazioni relative a prestazioni non documentate, un fattore di rischio che in ambito sanitario privato può tradursi in costi legali difficilmente quantificabili a priori ma potenzialmente rilevanti in caso di controversia.

Complessivamente, per una struttura di dimensioni medie come quella ipotizzata, l'adozione del sistema porterebbe a un risparmio operativo stimato tra i **2.000 e i 3.000 euro mensili**, tra tempo di segreteria liberato, riduzione degli errori di prenotazione e recupero parziale del fatturato perso per mancata presentazione, a fronte di un costo di gestione infrastrutturale del sistema (hosting, manutenzione, backup del database) stimabile in poche centinaia di euro mensili.

Applicazioni in altri settori: Lo stesso modello architetturale è applicabile in qualsiasi contesto che richieda la gestione di prenotazioni con risorse limitate nel tempo: studi legali, centri sportivi, saloni di bellezza, agenzie governative per appuntamenti al pubblico.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti):

Valutazione dei risultati

Potenzialità

L'elaborato raggiunge pienamente gli obiettivi fissati dalla Traccia PW 16, realizzando un servizio digitale completo e funzionalmente maturo per la gestione delle prenotazioni di un poliambulatorio. Di seguito analizzo in maniera più approfondita ciascuno dei punti di forza del progetto, motivandone la rilevanza tecnica e non limitandomi a una loro semplice enumerazione.

Il primo punto di forza riguarda la completezza funzionale del ciclo di vita gestito dall'applicazione. A differenza di molti progetti didattici che si limitano a implementare un

singolo processo isolato (ad esempio la sola prenotazione), il sistema copre l'intero percorso clinico e amministrativo di un paziente: dalla prenotazione autonoma della visita, passando per la conferma da parte del personale sanitario, fino alla conclusione dell'incontro clinico con la stesura di un referto strutturato e la sua eventuale condivisione controllata con altri professionisti della struttura. Questa ampiezza di copertura funzionale ha richiesto una progettazione del modello dati particolarmente attenta alla gestione degli stati e delle transizioni ammesse, come dimostra la macchina a stati implicita che governa il ciclo di vita di un appuntamento (`in_attesa`, `confermato`, `completato`, `cancellato`).

Il secondo punto di forza attiene alla qualità dell'esperienza utente offerta a ciascun ruolo. Ho progettato quattro interfacce distinte, ciascuna calibrata sulle esigenze specifiche dell'attore a cui è destinata, evitando l'errore, frequente in applicazioni gestionali di natura didattica, di offrire un'unica interfaccia generica adattata a posteriori a più ruoli tramite semplice occultamento condizionale di elementi grafici. Il wizard di prenotazione a più passaggi, sia nella versione destinata al paziente sia in quella destinata alla segreteria, riduce sensibilmente la probabilità di errore dell'utente, guidandolo attraverso un flusso lineare e reversibile.

Il terzo punto di forza riguarda la solidità architetturale ottenuta grazie alla netta separazione tra frontend e backend attraverso un'interfaccia REST ben definita. Questa scelta non è soltanto un requisito imposto dalla traccia, ma un principio ingegneristico che ho verificato concretamente durante lo sviluppo: la possibilità di modificare in maniera sostanziale l'interfaccia grafica di una dashboard, come avvenuto più volte nel corso delle otto fasi di sviluppo incrementale, senza alcuna necessità di intervenire sulla logica del backend sottostante, a riprova dell'effettivo disaccoppiamento tra i due livelli.

Il quarto punto di forza, particolarmente approfondito nella fase conclusiva del progetto, riguarda la postura di sicurezza dell'applicazione. Ho condotto un audit di sicurezza sistematico sull'intero backend, identificando e correggendo vulnerabilità di classe IDOR su molteplici endpoint sensibili (cancellazione appuntamenti, conferma check-in, aggiornamento note cliniche, gestione degli slot), introducendo controlli di titolarità (ownership check) verificati lato server a ogni richiesta. Ho inoltre implementato la sanitizzazione sistematica dell'output tramite la funzione `escapeHtml()` per prevenire attacchi di tipo Cross-Site Scripting, l'hashing delle password con la libreria `bcrypt`, un middleware per l'iniezione di header di sicurezza HTTP conformi alle raccomandazioni OWASP, e una validazione server-side rigorosa dei parametri in ingresso, che non si limita mai al solo controllo lato client.

Il quinto punto di forza, elemento distintivo rispetto a un tradizionale sistema di prenotazioni, riguarda la compliance sostanziale, e non meramente formale, al Regolamento Generale sulla Protezione dei Dati. La funzionalità del Dossier Sanitario Condiviso, unita all'informativa privacy pubblicata, al meccanismo di consenso esplicito e revocabile, e al cookie banner informativo, dimostra come i principi giuridici della minimizzazione dei dati e della privacy by design non siano stati trattati come un capitolo teorico separato, bensì tradotti in scelte implementative verificabili nel codice sorgente.

Il sesto punto di forza, infine, riguarda la flessibilità configurativa del sistema: la possibilità di personalizzare la durata dello slot per ciascun medico consente all'applicazione di adattarsi a specialità cliniche con tempistiche di visita profondamente differenti, da una consulenza rapida di quindici minuti a una prima visita specialistica di novanta minuti, senza alcuna necessità di modificare il codice sorgente ma agendo esclusivamente su un parametro di configurazione persistito nel database.

Limiti e possibili sviluppi futuri

Nonostante i risultati raggiunti, è opportuno riconoscere con onestà intellettuale i limiti attuali del sistema, alcuni dei quali derivano da scelte progettuali consapevoli operate per ragioni didattiche, altri costituiscono margini di miglioramento realistici in prospettiva di un'eventuale messa in produzione.

Il primo limite riguarda il meccanismo di autenticazione adottato, basato sulla persistenza dei dati dell'utente nel `localStorage` del browser anziché su un sistema di token firmati con scadenza (JSON Web Token) o su sessioni gestite lato server. Questa scelta, funzionale a comprendere in profondità il principio di statelessness del protocollo REST, comporta in un contesto di produzione reale un livello di sicurezza inferiore rispetto agli standard oggi raccomandati, poiché non prevede meccanismi di scadenza automatica della sessione né di revoca centralizzata di un accesso compromesso. Uno sviluppo naturale del progetto prevederebbe l'introduzione di un sistema di token JWT con refresh token, accompagnato da un meccanismo di blacklist per la revoca anticipata.

Il secondo limite riguarda l'assenza di un canale di notifica esterno al browser. Il sistema di check-in, pur funzionale, richiede che il paziente acceda attivamente alla propria dashboard per visualizzare la notifica pendente: in un contesto reale sarebbe opportuno affiancare a questo meccanismo un servizio di notifica push, un invio automatico di email tramite un server SMTP, oppure un'integrazione con servizi di messaggistica istantanea, ampliando la superficie di reperibilità della comunicazione verso il paziente.

Il terzo limite attiene alla persistenza dei dati a riposo. Sebbene siano già state implementate misure sostanziali di compliance GDPR quali il consenso esplicito, l'informativa privacy e la minimizzazione dei dati applicata al Dossier Sanitario Condiviso, il contenuto dei referti clinici viene attualmente memorizzato in chiaro nel database MySQL. Un'evoluzione naturale del sistema, per un contesto di produzione reale, prevederebbe l'introduzione di una cifratura simmetrica applicata al contenuto testuale dei referti prima della persistenza, con gestione sicura della chiave di cifratura separata dall'infrastruttura del database stesso.

Il quarto limite riguarda la scalabilità del livello dati in presenza di un volume di traffico significativamente superiore a quello testato. Sebbene abbia adottato accorgimenti quali l'uso sistematico di query con JOIN multiple per evitare il pattern N+1, un sistema destinato a servire un numero elevato di strutture sanitarie e di utenti concorrenti richiederebbe l'introduzione di indici aggiuntivi mirati sulle colonne più frequentemente interrogate, l'adozione di un pool di connessioni al database anziché l'apertura di una nuova connessione per ogni richiesta, e l'eventuale introduzione di un livello di caching (ad esempio tramite Redis) per i dati a bassa volatilità, quali il catalogo delle specialità.

Il quinto e ultimo limite riguarda l'assenza di una suite di test automatizzati. L'intera verifica funzionale del sistema, descritta nella sezione dedicata al test funzionale, è stata condotta manualmente, ripetendo sistematicamente ciascun flusso applicativo per ciascun ruolo dopo ogni modifica significativa al codice. Sebbene questo approccio abbia consentito di individuare e correggere efficacemente le anomalie riscontrate nel corso dello sviluppo, un contesto professionale richiederebbe l'introduzione di una suite di test automatizzati, comprendente test unitari sulla logica di validazione dei modelli Pydantic e test di integrazione sugli endpoint dell'API, eseguibili automaticamente a ogni modifica del codice sorgente tramite una pipeline di integrazione continua.